板

jimmyywang

2026 年 10 月 27 日

目录

1 数据结构 (Data Structures) 4
- 1.1 哈希表 . . . 4
- 1.2 fhq-treap . . . 4
- 1.3 RMQ (ST 表) . . . 6
- 1.4 笛卡尔树 (Cartesian Tree) . . . 7
- 1.5 树链剖分 (Heavy-Light Decomposition) . . . 7
- 1.6 LCT (Link-Cut Tree) . . . 8
- 1.7 李超线段树 . . . 10

2 图论 (Graph Theory) 13
- 2.1 连通性 (Connectivity) . . . 13
  - 2.1.1 Tarjan 求 SCC (Strongly Connected Components) . . . 13
  - 2.1.2 仅加边动态 scc . . . 13
  - 2.1.3 点双 / 边双 (Biconnected Components) . . . 14
  - 2.1.4 圆方树 (Round-Square Tree) . . . 16
- 2.2 最短路 (Shortest Path) . . . 17
  - 2.2.1 负环 (Negative Cycle Detection - SPFA) . . . 17
- 2.3 欧拉路径 (Eulerian Path) . . . 18
- 2.4 网络流 (Network Flow) . . . 18
  - 2.4.1 Dinic (Max Flow) . . . 18
  - 2.4.2 MCMF (Min Cost Max Flow) . . . 19
  - 2.4.3 网络流经典模型 . . . 21
- 2.5 图匹配 (Graph Matching) . . . 22
  - 2.5.1 二分图最小权完美匹配 (KM / SSP) . . . 22
- 2.6 树上算法 (Tree Algorithms) . . . 23
  - 2.6.1 树的重心 (Tree Centroid) . . . 23
  - 2.6.2 树上离某个点最远的点 . . . 24
  - 2.6.3 斯坦纳树 (Steiner Tree) . . . 24
- 2.7 2-SAT . . . 25

3 动态规划 (Dynamic Programming) 26
3.1 WQS 二分 . . . 26
3.2 决策单调性与二分队列 . . . 27

4 字符串 (String Algorithms) 28
4.1 字符串哈希 (String Hashing) . . . 28
4.2 KMP . . . 29
4.3 Manacher . . . 29
4.4 AC 自动机 (Aho-Corasick) . . . 29
4.5 后缀数组 (Suffix Array - SA) . . . 30
4.6 回文自动机 (Palindrome Automaton - PAM) . . . 31

5 数学 (Mathematics) 33
5.1 数论 (Number Theory) . . . 33
5.1.1 Exgcd (Extended Euclidean Algorithm) . . . 33
5.1.2 万能欧几里得 . . . 33
5.1.3 ExCRT (Extended Chinese Remainder Theorem) . . . 34
5.1.4 扩展欧拉定理 (Euler's Totient Theorem) . . . 34
5.1.5 Pollard-Rho . . . 34
5.1.6 Dirichlet 前缀和 (Dirichlet Prefix Sum) . . . 35
5.1.7 杜教筛 (Du's Sieve) . . . 36
5.1.8 Min25 . . . 37
5.1.9 单位根反演 . . . 39
5.2 组合数学 (Combinatorics) . . . 39
5.2.1 组合数 (Binomial Coefficients) . . . 39
5.2.2 Lucas 定理 (Lucas's Theorem) . . . 39
5.2.3 ExLucas (Extended Lucas Theorem) . . . 39
5.2.4 min-max 容斥 . . . 40
5.2.5 反演 . . . 41
5.3 线性代数 (Linear Algebra) . . . 41
5.3.1 高斯消元 (Gaussian Elimination) . . . 41
5.3.2 行列式 . . . 42
5.3.3 矩阵树定理 . . . 43
5.3.4 BEST 定理 (BEST Theorem) . . . 44
5.3.5 LGV 引理 (Weighted LGV Lemma) . . . 44
5.4 多项式 (Polynomials) . . . 44
5.4.1 Bostan-Mori . . . 49
5.5 数值方法 (Numerical Methods) . . . 49
5.5.1 自适应辛普森 (Adaptive Simpson's Rule) . . . 49

6 Misc 50
6.1 C++ 手动上下取整 . . . 50
6.2 单点修改区间摩尔投票 . . . 50
6.3 双栈模拟 dequeue . . . 50

6.4 测速 50
6.5 KKT 51

1 数据结构 (Data Structures)

1.1 哈希表

本地实现线性探测，yosupo 175ms（umap+ 同一个 shift 函数 334ms）

```
using u64=unsigned long long;
u64 xorshift(u64 x) {
  x += 0x9e3779b97f4a7c15;
    // x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
}
template<typename K, typename V, int M=(1<<20)>
struct HashTable{
  static_assert((M&(M-1))== 0);
    static const int MASK=M-1;
  int cnt=0;vector<int>e;
  bitset<M> used;
  K k[M]; V v[M];
  void clear(){
    for(auto x:e)used[x]=0;
    e.clear();cnt=0;
  }
  int size(){return cnt;}
  int idx(K x){
    int u=xorshift(x)&MASK;
    for(;used[u]&&k[u]!=x;(++u)&=MASK);
    return u;
  }
  V* find(K x){
    int i=idx(x);
    if(!used[i])return nullptr;
    return &v[i];
  }
  V& operator [](K x){
    int i=idx(x);
    if(!used[i]){
      used[i]=1;cnt++;
      e.pb(i);
      k[i]=x,v[i]=V{};
    }return v[i];
  }
};
```

然后当 std::map 用即可。注：此处默认 K, V 都是整数，如果有其他类型需要额外先写一个该类型的 hash。

多测与注意 每组调用 clear()。M 按键数留足空槽并取 2 的幂；此版本不处理表满，不支持直接删槽。

1.2 fhq-treap

1. 值域分裂

```
struct node{
    ll ls,rs;
    ll val,rnd;
```

```
    ll sz,num;
}t[1000010];ll cur=0;
ll newnode(ll val){
    t[++cur]={0,0,val,rand(),1,1};
    return cur;
}void upd(ll p){
    t[p].sz=t[t[p].ls].sz+t[t[p].rs].sz+t[p].num;
}void split(ll p,ll k,ll &x,ll &y){
    if(!p){x=y=0;return;}
    if(t[p].val<=k)x=p,split(t[p].rs,k,t[x].rs,y);
    else y=p,split(t[p].ls,k,x,t[y].ls);
    upd(p);
}ll merge(ll x,ll y){
    if(!x||!y)return x^y;
    if(t[x].rnd<t[y].rnd){
        t[x].rs=merge(t[x].rs,y);
        upd(x);return x;
    }else{
        t[y].ls=merge(x,t[y].ls);
        upd(y);return y;
    }
}
ll rt;
void init(){rt=cur=0;t[0]={};}
void insert(ll val){
    ll x,y,z;
    split(rt,val-1,x,y);split(y,val,y,z);
    if(y==0)y=newnode(val);else t[y].num++,t[y].sz++;
    rt=merge(merge(x,y),z);
}void del(ll val){
    ll x,y,z;
    split(rt,val-1,x,y);split(y,val,y,z);
    if(y==0){rt=merge(x,z);return;}t[y].num--,t[y].sz--;
    if(t[y].num==0)rt=merge(x,z);
    else rt=merge(merge(x,y),z);
}
ll rk(ll val){// 1 + (# of < val)
    ll x,y;split(rt,val-1,x,y);
    ll res=t[x].sz+1;rt=merge(x,y);
    return res;
}
ll findkth(ll p,ll k){
    while(1){
        if(t[t[p].ls].sz>=k)p=t[p].ls;
        else if(t[t[p].ls].sz+t[p].num>=k)return t[p].val;
        else k-=t[t[p].ls].sz+t[p].num,p=t[p].rs;
    }
}
ll pred(ll val){ // max_{x<val} x
    ll x,y;split(rt,val-1,x,y);
    ll res=findkth(x,t[x].sz);
    rt=merge(x,y);
    return res;
}ll succ(ll val){ // min_{x>val} x
    ll x,y;split(rt,val,x,y);
    ll res=findkth(y,1);
    rt=merge(x,y);
    return res;
}
```

2. 排名分裂

```
ll n,m,cnt;
struct node{
    int ls,rs;
    ll val,rnd;
    int sz,num;
    int tg;// 打了 tag，可以不用。这里是区间 reverse 板子。
}t[100010];
#define ls(p) (t[(p)].ls)
#define rs(p) (t[(p)].rs)
int newnode(ll x){
    t[++cnt]={0,0,x,rand(),1,1,0};
    return cnt;
}
void upd(int p){t[p].sz=t[ls(p)].sz+t[rs(p)].sz+t[p].num;}
void pd(int p){
    if(t[p].tg==0)return;
    swap(ls(p),rs(p));
    if(ls(p))t[ls(p)].tg^=1;
    if(rs(p))t[rs(p)].tg^=1;
    t[p].tg=0;
}
void split(int p,int sz,int &x,int &y){
    if(!p){x=y=0;return;}pd(p);
    if(t[ls(p)].sz<sz)x=p,split(rs(p),sz-t[ls(p)].sz-t[p].num,rs(p),y);
    else y=p,split(ls(p),sz,x,ls(p));
    upd(p);
}int merge(int x,int y){
    if(!x||!y)return x+y;pd(x),pd(y);
    if(t[x].rnd<t[y].rnd){rs(x)=merge(rs(x),y);upd(x);return x;}
    else {ls(y)=merge(x,ls(y));upd(y);return y;}
}
void out(int p){
    if(!p)return;pd(p);
    out(ls(p)),cout<<t[p].val<<" ",out(rs(p));
}
ll rt;
void init(){rt=cnt=0;t[0]={};}
```

多测与注意 两份代码独立选用，每组 init()。保留原 split、merge 和节点池；删除不回收编号。查询默认合法，val-1 要避开类型最小值；排名分裂版 num=1。

1.3 RMQ (ST 表)

ST 表 (Sparse Table)，用于 O(1) 查询区间最值 (RMQ)。

```
ll s[100010][20];
ll lg[100010];
ll get(ll l,ll r){
  ll L=r-l+1,Lg=lg[L];
  return max(s[l][Lg],s[r-(1<<Lg)+1][Lg]);
}
int main(){
  n=d,m=d;lg[1]=0;
  f(i,1,n)s[i][0]=d;
  f(j,1,19)f(i,1,n){
    if(i+(1<<(j-1))>n)s[i][j]=s[i][j-1];
```

```
    else s[i][j]=max(s[i][j-1],s[i+(1<<(j-1))][j-1]);
  }f(i,2,n)lg[i]=lg[i/2]+1;
}
```

多测与注意 按本组 n 重建表即可，不必清整表。数组和第二维按题目开，后者至少覆盖 $\log_2 n$；重叠合并需幂等，不能直接改成区间和。

1.4 笛卡尔树 (Cartesian Tree)

笛卡尔树 (Cartesian Tree)，一种基于键值和权值的二叉树，使用栈 O(n) 线性构建。

```
ll a[10000010];
ll l[10000010],r[10000010];
long long x,y;
ll st[10000010],tp;
int main(){
  n=d;tp=0;f(i,1,n)l[i]=r[i]=0;
    f(i,1,n)a[i]=d;
  st[++tp]=1;
  f(i,2,n){
    while(tp&&a[i]<a[st[tp]])l[i]=st[tp--];
    if(tp)r[st[tp]]=i;st[++tp]=i;
  }
}
```

多测与注意 已在读入后清本组左右儿子和栈顶；根是 st[1]，n 默认非零。结构下标可按题目改 int 省内存。

1.5 树链剖分 (Heavy-Light Decomposition)

树链剖分，将树上问题转化为序列问题，常配合线段树使用。

```
ll dfn[100010],de[100010],fa[100010],son[100010];
ll sz[100010],top[100010],rk[100010];
void dfs1(ll u,ll fat){
    fa[u]=fat;de[u]=de[fat]+1;sz[u]=1;son[u]=0;
    for(int i=hd[u];i;i=e[i].nx){
        ll v=e[i].v;if(v==fat)continue;
        dfs1(v,u);
        if(sz[v]>sz[son[u]])son[u]=v;
        sz[u]+=sz[v];
    }
}void dfs2(ll u,ll tp){
    top[u]=tp;dfn[u]=++tim;rk[tim]=u;
    if(son[u])dfs2(son[u],tp);
    for(int i=hd[u];i;i=e[i].nx){
        ll v=e[i].v;if(v==fa[u]||v==son[u])continue;
        dfs2(v,v);
    }
}
ll a[100010];
#define ls(p) (p<<1)
#define rs(p) (p<<1|1)
ll t[100010<<2];
ll tg[100010<<2];
void upd(ll p){t[p]=(t[ls(p)]+t[rs(p)])%mod;}
```

```
void build(ll l,ll r,ll p){
    tg[p]=0;
    if(l==r){t[p]=a[rk[l]];return;}
    ll mid=l+r>>1;
    build(l,mid,ls(p));
    build(mid+1,r,rs(p));
    upd(p);
}
void pd(ll p,ll l,ll r){
    if(tg[p]==0)return;ll mid=l+r>>1;
    tg[ls(p)]=(tg[ls(p)]+tg[p])%mod;
    tg[rs(p)]=(tg[rs(p)]+tg[p])%mod;
    t[ls(p)]=(t[ls(p)]+tg[p]*(mid-l+1))%mod;
    t[rs(p)]=(t[rs(p)]+tg[p]*(r-mid))%mod;
    tg[p]=0;
}
void add(ll l,ll r,ll p,ll L,ll R,ll k){
    if(L<=l&&r<=R){t[p]=(t[p]+k*(r-l+1))%mod;tg[p]=(tg[p]+k)%mod;return;}
    pd(p,l,r);
    ll mid=l+r>>1;
    if(mid>=L)add(l,mid,ls(p),L,R,k);
    if(mid<R)add(mid+1,r,rs(p),L,R,k);
    upd(p);
}ll get(ll l,ll r,ll p,ll L,ll R){
    if(L<=l&&r<=R)return t[p];
    pd(p,l,r);
    ll mid=l+r>>1,res=0;
    if(mid>=L)res+=get(l,mid,ls(p),L,R);
    if(mid<R)res+=get(mid+1,r,rs(p),L,R);
    return res%mod;
}
void chad(ll u,ll v,ll k){
    while(top[u]!=top[v]){
        if(de[top[u]]<de[top[v]])swap(u,v);
        add(1,n,1,dfn[top[u]],dfn[u],k);
        u=fa[top[u]];
    }if(dfn[u]>dfn[v])swap(u,v);
    add(1,n,1,dfn[u],dfn[v],k);
}ll chs(ll u,ll v){ll res=0;
    while(top[u]!=top[v]){
        if(de[top[u]]<de[top[v]])swap(u,v);
        res+=get(1,n,1,dfn[top[u]],dfn[u]);
        u=fa[top[u]];
    }if(dfn[u]>dfn[v])swap(u,v);
    res+=get(1,n,1,dfn[u],dfn[v]);
    return res%mod;
}
```

多测与注意 每组读边前清 hd、重置边计数，令 tim=0、sz[0]=de[0]=0，再 dfs1、dfs2、build。son 和 lazy 已在各自流程中重置；若有负增量，按题意归一化取模。

1.6 LCT (Link-Cut Tree)

反正我不会，看着乐呵乐呵好了。。

这里求得是链 xor

```
#include<bits/stdc++.h>
```

```
using namespace std;
#define ll int
#define f(i,a,b) for(ll i=a;i<=b;i++)
#define wt int tt=d;while(tt--)
#define py puts("Yes")
#define pn puts("No")
#define pritnf printf
#define fe(i,e) for(int i=0;i<e.size();i++)
#define vi vector<ll>
inline ll rd() {
  ll x=0,f=1;
  char c=getchar();
  while(!isdigit(c)){if(c=='-')f=-1;c=getchar();}
  while(isdigit(c))x=x*10+c-'0',c=getchar();
  return x*f;
}
ll dx[4]={0,1,0,-1};
ll dy[4]={1,0,-1,0};
#define d rd()
#define pb push_back
const ll N=300010;
struct edge{ll v,w,nx;}e[N<<1];
ll hd[N],cnt;
void add(ll u,ll v,ll w){e[++cnt]=(edge){v,w,hd[u]};hd[u]=cnt;}
ll qp(ll a,ll b,ll p){
  ll ans=1;while(b){
    if(b&1)ans=ans*a%p;
    a=a*a%p;b>>=1;
  }return ans;
}ll n,m;
ll Rt,tot;
ll fa[100010],ch[100010][2];
ll val[100010],sum[100010];
ll tg[100010];
#define ls(p) ch[p][0]
#define rs(p) ch[p][1]
bool isr(ll p){return (p!=ls(fa[p])&&p!=rs(fa[p]));}
void upd(ll p){sum[p]=sum[ls(p)]^sum[rs(p)]^val[p];}
bool get(ll p){return p==rs(fa[p]);}
void upt(ll p){swap(ls(p),rs(p));tg[p]^=1;}
void pd(ll p){
  if(!tg[p])return;
  if(ls(p))upt(ls(p));
  if(rs(p))upt(rs(p));
  tg[p]=0;
}void rot(ll x){
  ll y=fa[x],z=fa[y],t=get(x);
  if(!isr(y))ch[z][y==rs(z)]=x;
  ch[y][t]=ch[x][t^1];if(ch[x][t^1])fa[ch[x][t^1]]=y;
  ch[x][t^1]=y;fa[y]=x,fa[x]=z;upd(y);upd(x);
}void pda(ll x){if(!isr(x))pda(fa[x]);pd(x);}
void splay(ll x){//cout<<x<<endl;
  pda(x);//cout<<x<<endl;
  while(!isr(x)){
    ll y=fa[x],z=fa[y];
    if(!isr(y))rot(get(x)==get(y)?y:x);
    rot(x);
  }upd(x);
}void access(ll x){
```

```
  for(int y=0;x;y=x,x=fa[x]){
    splay(x);
    // cout<<x<<" "<<y<<endl;
    rs(x)=y,upd(x);
  }
}void mkr(ll x){
  access(x);
  splay(x);upt(x);
}ll find(ll x){
  access(x),splay(x);
  for(pd(x);ls(x);pd(x))x=ls(x);
  splay(x);return x;
}void split(ll x,ll y){
  mkr(x);access(y);splay(y);
}ll link(ll x,ll y){
  // cout<<x<<" "<<y<<endl;
  mkr(x);
  if(find(y)==x)return 0;
  fa[x]=y;splay(y);return 1;
}ll cut(ll x,ll y){
  split(x,y);
    if(ls(y)==x&&!rs(x)){ls(y)=fa[x]=0;upd(y);return 1;}return 0;
}
int main(){
  n=d,m=d;
    f(i,0,n)fa[i]=ch[i][0]=ch[i][1]=tg[i]=val[i]=sum[i]=0;
    f(i,1,n)sum[i]=val[i]=d;
  while(m--){
    ll o=d,x=d,y=d;
    if(o==0)split(x,y),printf("%d\n",sum[y]);
    if(o==1)link(x,y);if(o==2)cut(x,y);
    if(o==3)splay(x),val[x]=y,upd(x);
  }
  return 0;
}
```

多测与注意 main 的单组部分可放进 solve，初始化已补；仍是原来的数组版路径异或。查询默认两点连通，深链留意递归下传的栈空间。

1.7 李超线段树

平面加线段，查询 $x = x_0$ 相交的最低/高点。

```
//这份代码里，求的是相交的最低点，定义域为 [-1e9,1e9]，为了线段树是对的，整体向右偏移了 1e9。
const double eps=1e-9;
const int lim=1000000000;
const ll inf=0x3f3f3f3f3f3f3f3f;
int cmp(double x,double y){//比大小，这里求的是是否 x<y。如果要求最高点则这里反向就行。
    if(x-y>eps)return -1;
    if(y-x>eps)return 1;
    return 0;
}
struct line{double k,b;};
line li[400010];int cnt;
double calc(ll id,double x){//算点值
    if(id==0)return inf;
    line l=li[id];
    return l.k*x+l.b;
```

```
}
int nd;
int t[30000010];
int ls[30000010];
int rs[30000010];
void add(double x,double y,double xx,double yy){
    if(x>xx)swap(x,xx),swap(y,yy);
    cnt++;
    if(cmp(x,xx)==0){
        li[cnt].k=0,li[cnt].b=min(y,yy);//处理斜率不存在，直接退化成一个点。如果求相交最低点就是 min(y,yy)，否则是 max(y,yy)。
    }else{
        li[cnt].k=(yy-y)/(xx-x);
        li[cnt].b=y-li[cnt].k*x;
    }
}
//加入线段
void upd(int l,int r,int &p,int id){
    if(!p){p=++nd;t[p]=id;ls[p]=rs[p]=0;return;}
    int &oid=t[p],mid=((ll)l+r)>>1;
    int o=cmp(calc(id,mid-lim),calc(oid,mid-lim));
    if(o==1)swap(id,oid);
    if(l==r)return;
    if(cmp(calc(id,l-lim),calc(oid,l-lim))==1)upd(l,mid,ls[p],id);
    if(cmp(calc(id,r-lim),calc(oid,r-lim))==1)upd(mid+1,r,rs[p],id);
    // cout<<l<<" "<<r<<" "<<p<<" "<<t[p]<<endl;
}
//因为是线段所以需要找出所有相关区间，这里再找区间
void ch(int l,int r,int &p,int L,int R,int id){
    if(!p){p=++nd;t[p]=ls[p]=rs[p]=0;}
    // cout<<l<<" "<<r<<" "<<p<<endl;
    if(L<=l&&r<=R){
        upd(l,r,p,id);
        return;
    }int mid=((ll)l+r)>>1;
    if(mid>=L)ch(l,mid,ls[p],L,R,id);
    if(mid<R)ch(mid+1,r,rs[p],L,R,id);
}
//求相交最低点
double ask(int l,int r,int p,int x){
    // cout<<l<<" "<<r<<" "<<p<<" "<<x<<endl;
    if(!p)return inf;
    double res=t[p]==0?inf:calc(t[p],x-lim);
    // cout<<l<<" "<<r<<" "<<p<<" "<<x<<"   "<<t[p]<<" "<<res.x<<" "<<res.id<<endl;
    if(l==r)return res;
    int mid=((ll)l+r)>>1;
    if(mid>=x)res=min(res,ask(l,mid,ls[p],x));
    else res=min(res,ask(mid+1,r,rs[p],x));
    return res;
}
int n,m,q;
int rt;
void solve(){// 模板题处理的是输入 l,r,a,b，表示线段 y=ax+b,x\in [l,r)，通过 add 添加线段，ch进行更新，ask进行查找。
    rt=nd=cnt=0;
    n=d;q=d;
    f(i,1,n){
        ll l=d,r=d-1,a=d,b=d;
        ll x=l,xx=r;
```

```
        ll y=l*a+b,yy=r*a+b;
        add(x,y,xx,yy);
        // li[++cnt]={a,b};
        ch(0,2*lim,rt,l+lim,r+lim,cnt);
    }
    f(_,1,q){
        ll op=d;
        if(op==0){
            ll l=d,r=d-1,a=d,b=d;
            ll x=l,xx=r;
            ll y=l*a+b,yy=r*a+b;
            add(x,y,xx,yy);
            // li[++cnt]={a,b};
            ch(0,2*lim,rt,l+lim,r+lim,cnt);
        }if(op==1){
            ll x=d;
            double res=ask(0,2*lim,rt,x+lim);
            if(res==inf)puts("INFINITY");
            else printf("%lld\n",(ll)res);
        }
    }
}
```

多测与注意　保留浮点、静态节点池和原调用方式。每组重置 rt、nd、cnt，新节点的儿子和标号已清零。定义域按题目改；浮点比较和最后强转整数有精度风险，整数极大值题应改精确求值。

2 图论 (Graph Theory)

2.1 连通性 (Connectivity)

2.1.1 Tarjan 求 SCC (Strongly Connected Components)

Tarjan 算法，用于在线性时间内求解有向图的强连通分量。

```
ll dfn[100010],low[100010],cur;
ll bel[100010],scc;
ll st[100010],tp;
bool ins[100010];
vi e[100010];
void dfs(ll u){
  dfn[u]=low[u]=++cur;
  st[++tp]=u,ins[u]=1;
  fe(i,e[u]){
    ll v=e[u][i];
    if(!dfn[v])dfs(v),low[u]=min(low[u],low[v]);
    else if(ins[v])low[u]=min(low[u],dfn[v]);
  }if(low[u]==dfn[u]){
    scc++;ll now;while(1){
      now=st[tp--];
      ins[now]=0,bel[now]=scc;
      if(now==u)break;
    }
  }
}
void init(ll n){
    cur=scc=tp=0;
    f(i,1,n)dfn[i]=low[i]=bel[i]=ins[i]=0,e[i].clear();
}
```

多测与注意 每组 init(n) 后读边，遍历所有未访问点 dfs。bel 是逆拓扑编号；深链注意递归栈。

2.1.2 仅加边动态 scc

每条边的生效时间是不小于他的加入时间的，整体二分这个时刻，维护 (l, r, E) 表示生效时间在 $[l, r]$ 内的边。加入 $[l, mid]$ 中的边考察 scc 变化情况判断生效时间在左区间还是右区间（也可能不生效）。

```
int dfn[100010],low[100010],tim;
int st[100010],tp;
bool in[100010];
int bel[100010],scc;
vector<int>e[100010];
vector<int>node;
void dfs(int u){
    dfn[u]=low[u]=++tim;
    st[++tp]=u,in[u]=1;
    for(auto v:e[u]){
        if(!dfn[v])dfs(v),low[u]=min(low[u],low[v]);
        else if(in[v])low[u]=min(low[u],dfn[v]);
    }if(dfn[u]==low[u]){
        scc++;int now;while(1){
            now=st[tp--];
            bel[now]=scc;in[now]=0;
```

```
            if(now==u)break;
        }
    }
}
void clear(){
    tim=0,tp=0;scc=0;
    for(auto u:node)dfn[u]=low[u]=0,bel[u]=0,in[u]=0,e[u].clear();
    node.clear();
}
int U[100010],V[100010];
vector<int>con[100010];
int fa[100010];
int find(int x){
    if(fa[x]==x)return x;
    return fa[x]=find(fa[x]);
}
void un(int u,int v){
    if(find(u)==find(v))return;
    u=find(u),v=find(v);
    fa[u]=v;
}
void work(int l,int r,vector<array<int,3>>&E){// E 中储存 (u,v,i)，表示在加入全部 $[1,l-1]$ 的边后，u 和 v 所在的 scc 编号。这样接着在这个图上做可以简化。
    // printf("Working on [%d, %d]\n",l,r);
    // for(auto [u,v,i]:E)printf("(%d, %d, %d) ",u,v,i);cout<<endl;
    if(l>r||E.empty())return;
    int mid=(l+r)>>1;
    vector<array<int,3>> e1,e2;e1.clear(),e2.clear();
    for(auto [u,v,i]:E){
        node.pb(u),node.pb(v);
        if(i<=mid)e[u].pb(v);
    }
    for(auto u:node)if(!dfn[u])dfs(u);
    if(l==r){
        for(auto [u,v,i]:E){
            if(bel[u]==bel[v])un(U[i],V[i]);//有了“生效”状态后即可转化为并查集维护。
        }clear();
        return;
    }
    for(auto [u,v,i]:E){
        if(bel[u]!=bel[v])e2.pb({bel[u],bel[v],i});
        else if(i<=mid&&bel[u]==bel[v])e1.pb({u,v,i});
    }clear();
    work(l,mid,e1),work(mid+1,r,e2);
}
```

多测与注意 保留原整体二分片段。每组 fa[i]=i，清 node、dfn、low、in、bel、e，tim=tp=scc=0；U、V 与 E 重读。历史答案在叶子记录，返回后的 fa 是最终状态。

2.1.3 点双 / 边双 (Biconnected Components)

边双

```
// 注意：cnt初始值需要等于 1！不然异或失效！
ll dfn[500010],low[500010],tim;
bool cut[4000010];
void dfs(ll u,ll fa){
  dfn[u]=low[u]=++tim;
```

```
  for(int i=hd[u];i;i=e[i].nx){
    ll v=e[i].v;if(i==(fa^1))continue;
    // cout<<u<<" "<<v<<endl;
    if(!dfn[v]){
      dfs(v,i);low[u]=min(low[u],low[v]);
      if(dfn[u]<low[v])cut[i]=cut[i^1]=1;
    }else low[u]=min(low[u],dfn[v]);
  }
}ll fa[500010];
ll find(ll x){if(fa[x]==x)return x;return fa[x]=find(fa[x]);}
void un(ll u,ll v){fa[find(u)]=find(v);}
ll rep[500010],dcc;
vi Dcc[500010];
int main(){
  n=d,m=d;tim=dcc=0;cnt=1;
    f(i,1,n)hd[i]=dfn[i]=low[i]=rep[i]=0,Dcc[i].clear();
    f(i,0,2*m+2)cut[i]=0;
    f(i,1,m){
    ll u=d,v=d;
    add(u,v),add(v,u);
  }f(i,1,n)if(!dfn[i])dfs(i,0);
  // f(i,1,n)cout<<dfn[i]<<" "<<low[i]<<endl;
  f(i,1,n)fa[i]=i;
  f(i,2,cnt)if(!cut[i])un(e[i].u,e[i].v);
  f(i,1,n){
    if(!rep[find(i)])rep[find(i)]=++dcc;
    Dcc[rep[find(i)]].pb(i);
  }cout<<dcc<<endl;
  f(i,1,dcc){
    cout<<Dcc[i].size()<<" ";
    fe(j,Dcc[i])cout<<Dcc[i][j]<<" ";
    cout<<endl;
  }
  return 0;
}
```

点双

```
vi e[500010];
ll dfn[500010],low[500010],tim;
ll st[500010],tp,dcc;
bool in[500010];
vi Dcc[500010];
vector<array<int,2>>ed[1000010];
ll deg[500010];
array<int,2>ste[1000010];
int tpe;
void dfs(int u,int fa){
    dfn[u]=low[u]=++tim;
    st[++tp]=u;in[u]=1;
    bool skipped=0;
    for(auto v:e[u]){
        if(v==u)continue;
        if(v==fa&&!skipped){skipped=1;continue;}
        if(!dfn[v]){
            ste[++tpe]={u,v};
            dfs(v,u);if(dfn[u]<=low[v]){
                dcc++;Dcc[dcc].pb(u);
                ll now;
```

```
                while(1){
                    now=st[tp--];in[now]=0;
                    Dcc[dcc].pb(now);
                    if(now==v)break;
                }array<int,2>E;
                while(1){
                    E=ste[tpe--];
                    ed[dcc].pb(E);
                    if(E==(array<int,2>){u,v})break;
                }
            }low[u]=min(low[u],low[v]);
        }else if(in[v]){
            low[u]=min(low[u],dfn[v]);
            if(dfn[v]<dfn[u])ste[++tpe]={u,v};
        }
    }
}
int main(){
  n=d,m=d;tim=tp=tpe=0;
    f(i,1,dcc)Dcc[i].clear(),ed[i].clear();dcc=0;
    f(i,1,n)e[i].clear(),dfn[i]=low[i]=deg[i]=in[i]=0;
    f(i,1,m){
    ll u=d,v=d;if(u!=v)deg[u]++,deg[v]++;
    e[u].pb(v),e[v].pb(u);
  }f(i,1,n)if(!dfn[i]){
        if(deg[i]==0)Dcc[++dcc].pb(i);
        else tp=0,tpe=0,dfs(i,0);
    }
  cout<<dcc<<endl;
  f(i,1,dcc){
    cout<<Dcc[i].size()<<" ";
    fe(j,Dcc[i])cout<<Dcc[i][j]<<" ";
    cout<<endl;
  }
  return 0;
}
```

Note: 如果需要逐点双遍历（如点双内 dfs），必须把点双内的边集也求出！不然复杂度错误！一个 trivial 的做法是往边上赛一个点，求出点双后如果集合内包含一个边上塞的点，那么这条边在点双内。这种做法在点双大小为 2 的时候会挂，比较难处理。

现已将代码改为可以直接求出边集。

多测与注意 两份代码独立选用，已按本组清初始化。边双 cnt 从 1 起，按父边编号跳过反边以支持重边；点双仅跳一条父边。点双内部遍历用 ed，别反复扫原邻接表。

2.1.4 圆方树 (Round-Square Tree)

圆方树 (Block-Cut Tree)，用于处理一般图上的点双连通分量。

```
ll n,m,tim;
ll dfn[100010],low[100010];
ll st[100010],tp,c;
bool in[100010];
vi e[100010];
vi E[200010];
void dfs(ll u,ll fa){
  dfn[u]=low[u]=++tim;
```

```
  st[++tp]=u;in[u]=1;
  bool skipped=0;
  fe(i,e[u]){
    ll v=e[u][i];
        if(v==fa&&!skipped){skipped=1;continue;}
    if(!dfn[v]){
      dfs(v,u);low[u]=min(low[u],low[v]);
      if(low[v]>=dfn[u]){
        c++;E[c].pb(u),E[u].pb(c);
        ll now;while(1){
          now=st[tp--];in[now]=0;
          E[now].pb(c),E[c].pb(now);
          if(now==v)break;
        }
      }
    }else if(in[v])low[u]=min(low[u],dfn[v]);
  }
}

int main(){
  n=c=d,m=d;tim=tp=0;
    f(i,1,n)e[i].clear(),dfn[i]=low[i]=in[i]=0;
    f(i,1,2*n)E[i].clear();
  f(i,1,m){ll u=d,v=d;e[u].pb(v),e[v].pb(u);}
  f(i,1,n)if(!dfn[i]){dfs(i,0);tp=0;in[i]=0;}
  return 0;
}
```

多测与注意 保留原顶点栈写法，已补多测清空并只跳一条父边。非连通图得到森林；方点从 n+1 编号。

2.2 最短路 (Shortest Path)

2.2.1 负环 (Negative Cycle Detection - SPFA)

使用 SPFA (Shortest Path Faster Algorithm) 检测图中的负环。

```
ll dis[10010];
bool in[10010];
ll c[10010];
queue<ll>q;
bool work(){
  while(!q.empty())q.pop();
    f(i,1,n)dis[i]=c[i]=0,in[i]=1,q.push(i);
  while(!q.empty()){
    ll u=q.front();q.pop();in[u]=0;
    for(int i=hd[u];i;i=e[i].nx){
      ll v=e[i].v;if(dis[v]>dis[u]+e[i].w){
        dis[v]=dis[u]+e[i].w;c[v]=c[u]+1;
                if(c[v]>=n)return 1;
        if(!in[v]){
          in[v]=1;
          q.push(v);
        }
      }
    }
  }return 0;
}
```

多测与注意 每次 work 自行重置距离、计数和队列；图按组重建。默认检测全图，若只查源可达负环则仅从源入队。SPFA 最坏 $O(nm)$。

2.3 欧拉路径 (Eulerian Path)

有向图

```
vi e[100010];
ll cur[100010];
ll in[100010],out[100010];
ll ans[1000010],c;
void dfs(ll u){
  for(int i=cur[u];i<e[u].size();i=cur[u]){
    ll v=e[u][i];cur[u]=i+1;
    dfs(v);
  }ans[++c]=u;
}
int main(){
  n=d,m=d;c=0;
    f(i,1,n)e[i].clear(),cur[i]=in[i]=out[i]=0;
    f(i,1,m){
    ll u=d,v=d;
    e[u].pb(v);
    in[v]++,out[u]++;
  }f(i,1,n)sort(e[i].begin(),e[i].end());
  ll x = -1, y = -1; // x: start, y: end
  f(i,1,n){
    // if(i==1)cout<<in[i]<<" "<<out[i]<<endl;
    if(in[i]==out[i])continue;
    if(abs(in[i]-out[i])>1){pn;return 0;}
    if(in[i]==out[i]+1){if(y!=-1){pn;return 0;}y=i;}
    if(in[i]==out[i]-1){if(x!=-1){pn;return 0;}x=i;}
  }if(x==-1){x=1;f(i,1,n)if(out[i]){x=i;break;}}dfs(x);
    if(c!=m+1){pn;return 0;}
  for(int i=c;i>=1;i--)cout<<ans[i]<<" ";
  return 0;
}
```

无向图 无向图：把经过过的边标记一下不要走即可 (Hierholzer 算法)。

多测与注意 保留递归 Hierholzer，已补非零度起点与用尽所有边的检查。多测初始化已放在读边前；边很多时 DFS 栈深为边数。

2.4 网络流 (Network Flow)

2.4.1 Dinic (Max Flow)

Dinic 算法，用于高效求解网络最大流。

```
struct node{ll u,v,w,nx;}e[1000010];
ll hd[100010],cnt=1;
void add(ll u,ll v,ll w){e[++cnt]={u,v,w,hd[u]};hd[u]=cnt;}
void ad(ll u,ll v,ll w){add(u,v,w),add(v,u,0);}

ll de[100010],cur[100010];
ll dis[100010];
const ll inf=LLONG_MAX/4;
```

```
queue<ll>q;
ll nodes,S,T;
bool bfs(){
    while(!q.empty())q.pop();
    f(i,1,nodes)cur[i]=hd[i],de[i]=inf;de[S]=0;q.push(S);
    while(!q.empty()){
        ll u=q.front();q.pop();
        for(int i=hd[u];i;i=e[i].nx){
            ll v=e[i].v;if(e[i].w==0)continue;
            if(de[v]==inf){
                de[v]=de[u]+1;q.push(v);
                if(v==T)return 1;
            }
        }
    }return 0;
}
ll dfs(ll u,ll fl){
    if(u==T)return fl;
    ll res=0,k=0;
    for(ll &i=cur[u];i&&fl;i=e[i].nx){if(e[i].w==0)continue;
        ll v=e[i].v;if(de[v]==de[u]+1){
            k=dfs(v,min(fl,e[i].w));
            e[i].w-=k,e[i^1].w+=k;
            fl-=k,res+=k;
            if(!fl)break;
        }
    }return res;
}
ll n,m;
ll dinic(){
    ll res=0;
    if(S==T)return 0;
    while(bfs())res+=dfs(S,inf);
    return res;
}
int main(){
    n=d,m=d,S=d,T=d;nodes=n;cnt=1;fill(hd,hd+nodes+1,0);
    f(i,1,m){
        ll u=d,v=d,w=d;
        ad(u,v,w);
    }cout<<dinic();
    return 0;
}
```

多测与注意 读边前已清 hd、cnt=1；保留原接口。容量非负，nodes 含超级源汇，边池按正反两倍开。再跑一次返回新增流量。

2.4.2 MCMF (Min Cost Max Flow)

upd on 25.10.12 已严肃进行势能优化 (使用 SPFA 预处理 h[ ] 数组，后续使用 Dijkstra)。

```
struct node{ll u,v,w,c,nx;}e[1000010];
ll hd[100010],cnt=1;
void add(ll u,ll v,ll w,ll c){e[++cnt]={u,v,w,c,hd[u]};hd[u]=cnt;}
void ad(ll u,ll v,ll w,ll c){add(u,v,w,c),add(v,u,0,-c);}
ll n,m,S,T,nodes;
ll cst;
ll pre[100010];
```

```
ll dis[100010];
ll h[100010];
const ll inf=LLONG_MAX/4;
ll arg(){
    ll u,v=T,fl=inf;
    while(v!=S)u=pre[v],fl=min(fl,e[u].w),v=e[u].u;
    v=T;while(v!=S)u=pre[v],e[u].w-=fl,e[u^1].w+=fl,v=e[u].u;
    cst+=fl*(dis[T]-(h[S]-h[T]));return fl;
}
struct myp{ll w,x;};
bool operator < (myp a,myp b){return a.w>b.w;}
bool in[100010];
bool dij(){
    priority_queue<myp>q;
    f(i,1,nodes)dis[i]=inf,pre[i]=in[i]=0;
    dis[S]=0;q.push({0,S});
    while(!q.empty()){
        ll u=q.top().x;q.pop();
        if(in[u])continue;in[u]=1;
        for(int i=hd[u];i;i=e[i].nx){
            if(e[i].w==0)continue;ll v=e[i].v;
            if(dis[u]+(e[i].c+h[u]-h[v])<dis[v]){
                dis[v]=dis[u]+(e[i].c+h[u]-h[v]);pre[v]=i;
                q.push({dis[v],v});
            }
        }
    }
    return dis[T]!=inf;
}
array<ll,2> mcmf(){
    ll fl=0;
    while(dij()){
        fl+=arg();
        f(i,1,nodes)if(dis[i]!=inf)h[i]+=dis[i];
        // f(i,1,nodes)cout<<h[i]<<" ";cout<<endl;
    }return {fl,cst};
}
void spfa(ll s){
    queue<ll>q;
    fill(h+1,h+nodes+1,inf);
    fill(in,in+nodes+1,0);
    h[s]=0;q.push(s);in[s]=1;
    while(!q.empty()){
        ll u=q.front();q.pop();in[u]=0;
        for(int i=hd[u];i;i=e[i].nx){
            if(e[i].w==0)continue;ll v=e[i].v;
            if(h[u]+e[i].c<h[v]){
                h[v]=h[u]+e[i].c;pre[v]=i;
                if(!in[v])in[v]=1,q.push(v);
            }
        }
    }
}
int main(){
    n=d,m=d,S=d,T=d;nodes=n;cnt=1;cst=0;fill(hd,hd+nodes+1,0);
    f(i,1,m){
        ll u=d,v=d,w=d,c=d;
        ad(u,v,w,c);
    }spfa(S);
```

```
67      auto res=mcmf();
68      cout<<res[0]<<" "<<res[1]<<endl;
69      return 0;
70  }
```

多测与注意 每组重建图并 cst=0，读完后 spfa(S) 再 mcmf。已避免给不可达点累加 INF；初始源可达负费用环需另行处理，INF 与费用类型按题目选。

2.4.3 网络流经典模型

上下界无费用流

1. 无源汇上下界可行流：钦定容量下界流满并计算这种情况下每个点的净流入量 in_u，新建超级源点 S 和超级汇点 T 用以模拟净流入的影响。如果 $in_u > 0$ 则连边 (S, u, in_u)，否则连边 $(u, T, -in_u)$。原图中其他边容量改为上下界之差。跑新图最大流，若满流则有可行流，把每条边原来的下界加上即可；否则无解。

2. 有源汇上下界可行流：新建一条 $(T, S, +\infty)$ 的边即转化为无源汇上下界可行流。

3. 有源汇上下界最大流：跑完无源汇的可行流后，断掉 $(T, S, +\infty)$ 的边后，残量网络上源汇之间（注意是原来的源汇不是新建的）继续跑最大流，加上即可。

4. 有源汇上下界最小流：跑完无源汇的可行流后，断掉 $(T, S, +\infty)$ 的边后，残量网络上汇点到源点跑最大流，即求最大退流，减去即可。

上下界费用流

以最小费用为例。若要求费用最大，全体费用取相反数以后做即可。鉴于可能出现负环，对于负权费用流后文会提到处理方式。以下默认费用为正。

1. 无源汇上下界最小费用可行流：同无费用版本，把每一步都算上费用，最大流换成最小费用最大流。

   注：费用全正的情况下，不需要跑最小费用退流。如果能退，则存在权为负的增广路，没理由不在最小费用最大流的时候被处理过。

2. 有源汇上下界最小费用可行流：新建一条 $(T, S, +\infty, 0)$ 的边即转化为无源汇上下界最小费用可行流。

3. 有源汇上下界最小费用最大流：跑完无源汇的最小费用可行流后，断掉 $(T, S, +\infty, 0)$ 的边后，残量网络上源汇之间继续跑最小费用最大流，加上即可。

4. 有源汇上下界最小费用最小流：跑完无源汇的最小费用可行流后，断掉 $(T, S, +\infty, 0)$ 的边后，残量网络上汇点到源点跑最小费用最大流，减去即可（费用是加上）。

负权费用流

其实没那么神秘。对于每条负权边 (u, v, c, w)，我们把它变成两条边：一条是 (u, v, c, w) 强制满流，另一条是 $(v, u, c, -w)$ 的退流边。强制满流直接当作上下界限制即可，在跑新图的时候负权边的容量会变成 0，等于没有这条边。这样所有费用都是正的。

同余网络流

现编的名字，即每条边流量都有一个全局的 $\bmod k$ 限制。把每条边的流量写成 $kx+y$，那么每个点处就会有关于 x 的流量平衡，由于 y 的净流入可能是 k 的正数倍 Δk，所有流量平衡有一个偏差项 Δ，与上下界一样用额外源汇模拟偏差即可在 x 上做流子。

最大权闭合子图

建立源汇 S,T，对正权点连边 (S,u,w_u)，负权点连边 $(u,T,-w_u)$，原图边 (u,v) 建 (u,v,∞)，全图正权和减去最小割即为答案。

答案构造：最大流后的残量图中，从源点沿正残量边可达的原图顶点组成所选闭合子图。

最长反链

对原图每个点拆点 (in_u,out_u)，原图边 (u,v) 改为 (out_u,in_v)，所有 in_u 点权为 -1，所有 out_u 点权为 1。此图最大权闭合子图即为最长反链。

这个图比较好理解，因为一个最大的闭合子图一定是由若干个 out_u 作为起点发出的（否则权值和非正），这几个 u 构成了一个反链。

多测与注意 每组清网络、需求和基准费用。可行流后删超级源汇关联边和额外 $T\to S$ 边的正反两向，先记录基准流；最小非负流的退流不超过基准流。反链模型需覆盖传递可达关系，可补 in 到 out 的无穷边。

2.5 图匹配 (Graph Matching)

2.5.1 二分图最小权完美匹配 (KM / SSP)

Kuhn-Munkres (KM) 算法，用于 O(n^3) 求解二分图最小/大权完美匹配。

```
ll cst[1010][1010];
#define inf 0x3f3f3f3f3f3f3f3f
void KM(int n,int m){
    vector<ll>l(n+1),r(m+1);
    vector<int>p(m+1),ans(n+1);
    f(i,1,n){
        vector<ll>dis(m+1,inf);
        vector<int>pre(m+1),done(m+1);
        int x,v=0,u=0;
        for(p[0]=i;x=p[u];u=v){
            done[u]=1;
            ll mn=inf;
            f(j,1,m)if(!done[j]){
                ll w=cst[x][j]-l[x]-r[j];
                if(w<dis[j])dis[j]=w,pre[j]=u;
                if(dis[j]<mn)mn=dis[j],v=j;
            }
            f(j,0,m){
                if(done[j])l[p[j]]+=mn,r[j]-=mn;
                else dis[j]-=mn;
            }
        }
        for(int v;u;u=v)v=pre[u],p[u]=p[v];
    }f(j,1,m)ans[p[j]]=j;
    // cout<<(-r[0])<<endl;//最小权
    // f(j,1,m)cout<<p[j]<<" ";cout<<endl;//右部点的匹配点。左部点匹配点是 ans。
}
```

另一种费用流做法（SSP） 板子题没 KM 快。$O(|match|(|V|^2+|E|))$。

```cpp
#define inf 0x3f3f3f3f3f3f3f3f
vector<pair<int,ll>>e[1010];//左边第 u 个点和右边第 v 个点，权为 w，建边 (u,{v,w})
bool down(ll &x,ll y){return x>y?x=y,1:0;}
void KM(int n,int m){
    vector<ll>h(n+1),rv(m+1);
    vector<int>R(m+1);
    ll cost=0;
    for(;;){
        vector<ll>dis(n+1);
        vector<int>vis(n+1),p0(n+1),p1(m+1);
        f(i,0,m)dis[R[i]]=inf;
        for(int x=0;vis[x]^=1;){
            ll mn=inf;
            for(auto [j,w]:e[x])if(down(dis[R[j]],dis[x]+w-rv[j]))p0[R[j]]=j,p1[j]=x;
            f(j,1,n)if(!vis[j]&&down(mn,dis[j]-h[j]))x=j;
        }if(dis[0]>=inf/2)break;
        for(int r=p0[0];r;r=p0[R[r]]){
            rv[r]+=dis[R[r]]-dis[p1[r]];
            R[r]=p1[r];
        }cost+=dis[0],h=dis;

        // cout<<cost<<endl;
        // for(auto _:R)cout<<_<<" ";cout<<endl;
        // for(auto _:rv)cout<<_<<" ";cout<<endl;
    }
    //cost: 最小权
    //R: 右部点的匹配点
    //rv: 右部点匹配边的权值
    //注意到每次增广一条路径，匹配数增加 1，意味着可以得到恰有多少个匹配时候的最小权
    // cout<<cost<<endl;
    // f(i,1,m)cout<<R[i]<<" ";cout<<endl;
}
```

多测与注意 两份 KM 均保留原写法、原局部结果变量。第一份需 $n \le m$，每组重填 cst；第二份读边前清 e，得到最大匹配数下的最小费用，不保证完美匹配。

2.6 树上算法 (Tree Algorithms)

2.6.1 树的重心 (Tree Centroid)

从点分模板贺的，所以有 fl 数组表示有没有被处理过。

```cpp
ll sz[100010],g,mn;
ll f[100010];
bool fl[100010];
ll siz;
void dfsz(ll u,ll fa){if(fl[u])return;
  f[u]=fa;siz++;
  sz[u]=1;for(int i=hd[u];i;i=e[i].nx){
    ll v=e[i].v,w=e[i].w;
    if(v==fa||fl[v])continue;
    dfsz(v,u),sz[u]+=sz[v];
  }
}
void dfsg(ll u,ll fa){if(fl[u])return;
  ll mx=0;
  if(f[u]!=0)mx=siz-sz[u];
  for(int i=hd[u];i;i=e[i].nx){
```

```
    ll v=e[i].v,w=e[i].w;
    if(v==fa||fl[v])continue;
    dfsg(v,u),mx=max(mx,sz[v]);
  }if(mx<mn){mn=mx,g=u;}
}
void getg(ll rt){
  mn=0x7f7f7f7f;siz=0;g=0;
  dfsz(rt,0),dfsg(rt,0);
}
```

多测与注意 每组重建图并清 fl；同组点分治递归不能清 fl。getg 已自行重置统计量。

2.6.2 树上离某个点最远的点

mx: 往子树去的最远（不包括自身）

far：往父亲去的最远（包括自身）

```
ll mx[200010],imx[200010],id[200010];
ll dep[200010];
ll far[200010];
void getd(ll u,ll fa){
    dep[u]=dep[fa]+1;mx[u]=imx[u]=0;id[u]=0;
    for(int i=0;i<e[u].size();i++){
        ll v=e[u][i];if(v==fa)continue;
        getd(v,u);if(mx[v]+1>mx[u]){
            imx[u]=mx[u];mx[u]=mx[v]+1;id[u]=v;
        }else if(mx[v]+1>=imx[u]){
            imx[u]=max(imx[u],mx[v]+1);
        }
    }
}
void getf(ll u,ll fa,ll mxl){
    if(u==rt){far[u]=1;
        for(int i=0;i<e[u].size();i++){
            ll v=e[u][i];if(v==fa)continue;
            getf(v,u,(v==id[u]?imx[u]:mx[u])+1);
        }
    }else{
        far[u]=mxl+1;
        for(int i=0;i<e[u].size();i++){
            ll v=e[u][i];if(v==fa)continue;
            if(v==id[u])getf(v,u,max(mxl+1,imx[u]+1));
            else getf(v,u,max(mxl+1,mx[u]+1));
        }
    }
}
```

多测与注意 mx 为向下的边数，far 为向上的点数；答案取 mx[u]+1 与 far[u] 的较大者。先 getd，再 getf(rt,0,0)，每组重建图。

2.6.3 斯坦纳树 (Steiner Tree)

斯坦纳树 (Steiner Tree)，用于在图中连接指定的一组“关键点”，使得总边权最小。

```
ll n,m,s;
ll dis[110][110];
```

```cpp
ll id[110],a[110];
ll dp[110][1500];
bool fl[110];
struct node{ll x,id;};
bool operator <(node a,node b){return a.x>b.x;}
priority_queue<node>q;
ll res=0x3f3f3f3f3f3f3f3f;
int main(){
  n=d,m=d,s=d;res=0x3f3f3f3f3f3f3f3f;
    while(!q.empty())q.pop();fill(id,id+n+1,0);
  memset(dis,0x3f,sizeof(dis));
    f(i,1,n)dis[i][i]=0;
  f(i,1,m){
    ll u=d,v=d,w=d;
    dis[u][v]=dis[v][u]=min(dis[u][v],w);
  }f(i,1,s)id[a[i]=d]=i;
  f(k,1,n)f(i,1,n)f(j,1,n)dis[i][j]=min(dis[i][j],dis[i][k]+dis[k][j]);
  memset(dp,0x3f,sizeof(dp));
  f(i,1,n)dp[i][0]=0;
  f(i,1,s)dp[a[i]][1<<(i-1)]=0;
  f(S,1,(1<<s)-1){
    f(i,1,n)for(int t=S;t;t=(t-1)&S)dp[i][S]=min(dp[i][S],dp[i][t]+dp[i][S^t]);
    f(i,0,n)fl[i]=0;
    dp[0][S]=0;q.push({0,0});
    while(!q.empty()){
      ll u=q.top().id;q.pop();
      if(fl[u])continue;fl[u]=1;
      if(u==0)f(i,1,n)q.push({dp[i][S],i});
      else{
        f(i,1,n)if(dp[u][S]+dis[u][i]<dp[i][S]){
          dp[i][S]=dp[u][S]+dis[u][i];
          q.push({dp[i][S],i});
        }
      }
    }
  }f(i,1,n)res=min(res,dp[i][(1<<s)-1]);
  cout<<res;
  return 0;
}
```

多测与注意 保留 Floyd 加子集 DP；多测已重置 res、队列、id。边权非负，状态数组按终端数开；终端数为 0 时答案另取 0，不可达时按题目输出。

2.7 2-SAT

2-SAT (2-Satisfiability) 问题。下面是基于 Tarjan SCC 的解法。scc[i] 表示 i 所属的 scc 是哪个。原理是 tarjan 会自动求出倒序的拓扑序。

```cpp
f(i,1,n)if(scc[i]==scc[i+n]){puts("IMPOSSIBLE");return 0;}
f(i,1,n)printf("%lld ",scc[i]>scc[i+n]);
//scc[i]表示i所属的scc是哪个。原理是tarjan会自动求出倒序的拓扑序
```

多测与注意 保留原赋值式，约定 i 表示假、i+n 表示真；原 Tarjan 的分量数组 bel 对应此处 scc。每组初始化 2n 个点及蕴含图。

3 动态规划 (Dynamic Programming)

3.1 WQS 二分

WQS 二分（Alien Trick）用于“恰好选 m 个”的最优化问题。设原 DP 的一个方案代价为 V，使用了 c 段（或选了 c 个物品），给每一段附加代价 k 后，转而最小化

$$V + kc.$$

当 k 增大时，最优方案使用的段数 $c(k)$ 单调不增，因此可以二分 k，把一维“段数”从 DP 状态中消掉。若目标为恰好 m 段，在边界处有

$$\text{ans} = F_k(n) - km.$$

适用时需要确认两件事：一是加权后的最优段数关于 k 单调；二是要恢复的 (c, V) 点确实位于这些方案的下凸包上，也就是关于段数的最优值具有所需的离散凸性。第二点不能只由“段数单调”推出，需要结合题目的费用结构证明；很多满足四边形不等式的分段 DP 会出现这种结构。整数代价下通常直接二分整数 k；若需要非整数斜率，则按题目精度扩大或改为实数处理。

实现中 DP 同时记录 $(\text{cost}, \text{cnt})$。求最小值时若加权代价相同，通常取更小的 cnt，这样 $c(k)$ 在二分边界处定义稳定。下面是可复用核心：

```
struct State{
    ll v; int cnt;
};
State dp[N];

bool better(State a,State b){
    if(a.v!=b.v)return a.v<b.v;
    return a.cnt<b.cnt;              // min problem: tie -> fewer groups
}

int check(ll k){
    dp[0]={0,0};
    for(int i=1;i<=n;i++){
        dp[i]={INF,0};
        for(int j=0;j<i;j++){
            State t={dp[j].v+cost(j+1,i)+k,dp[j].cnt+1};
            if(better(t,dp[i]))dp[i]=t;
        }
    }
    return dp[n].cnt;
}

ll wqs(int m,ll L,ll R){
    while(L<R){
        ll mid=(L+R)>>1;
        if(check(mid)<=m)R=mid;
        else L=mid+1;
    }
    check(L);
    return dp[n].v-L*m;
}
```

上面只展示 WQS 本身，`check` 内部仍是 $O(n^2)$；实际使用时再套斜率优化、决策单调性等优化即可。

多测与注意 保留原 DP 与二分。每次 check 已覆盖 dp，本组 cost 预处理另建。需离散凸性，二分边界按题目选。

3.2 决策单调性与二分队列

考虑一类一维分段 DP

$$f_i = \min_{0 \le j < i} \{f_j + w(j+1, i)\}.$$

若转移矩阵是 Monge 的，即对 $j < J \le i < I$ 有四边形不等式

$$w(j+1, i) + w(J+1, I) \le w(j+1, I) + w(J+1, i),$$

则最优决策点 $p_i = \operatorname{argmin}_j \{f_j + w(j+1, i)\}$ 单调不减。加上只与 j 有关的 f_j 不会破坏 Monge 性。

对区间费用 $C(l, r)$，常写成

$$C(a, c) + C(b, d) \le C(a, d) + C(b, c), \qquad a \le b \le c \le d.$$

在完整的二维定义域上，也可以只检查相邻的 2×2 条件

$$C(l, r) + C(l+1, r+1) \le C(l, r+1) + C(l+1, r),$$

它等价于所有四边形不等式。实际题目中通常直接从费用的结构证明；例如有序点分段后到中位数的 L_1 代价就是常见的 Monge 费用。

二分队列维护若干候选决策 (j, L)，表示从位置 L 开始，j 是当前队列中最优的决策。由于决策单调，两个候选的优劣只会发生一次切换：插入新决策 i 时，与队尾旧决策比较，并二分出新决策第一次不劣的位置；若新决策从队尾的起点起就更优，则直接弹掉队尾。查询 f_i 时只需根据这些分界点移动队首。每个决策至多入队、出队一次，另有一次二分，因此总复杂度 $O(n \log n)$。

下面是最小化问题的核心模板。`better(j,J,x)` 表示在终点 x 处决策 j 是否严格优于 J；相等时的 tie-break 要与 WQS 中记录的段数保持一致。

```
ll dp[N];
int cnt[N];
struct Node{ int j,l; } q[N];
int hd,tl;

bool better(int j,int J,int i){
    ll x=dp[j]+cost(j+1,i);
    ll y=dp[J]+cost(J+1,i);
    if(x!=y)return x<y;
    return cnt[j]<cnt[J];
}

void take(int j,int i,ll extra=0){
    dp[i]=dp[j]+cost(j+1,i)+extra;
    cnt[i]=cnt[j]+1;
}

void insert_candidate(int i){
    if(i==n)return;
    while(hd<=tl){
        int j=q[tl].j;
        int l=max(i+1,q[tl].l),r=n;

        // old candidate is still better at n: i never becomes optimal
```

```
        if(better(j,i,n))return;

        // first position where i is no worse than j
        while(l<r){
            int mid=(l+r)>>1;
            if(better(j,i,mid))l=mid+1;
            else r=mid;
        }

        if(l==q[tl].l)--tl;          // i dominates the whole last interval
        else{
            q[++tl]={i,l};
            return;
        }
    }
    q[++tl]={i,i+1};
}

void solve_dp(ll extra=0){
    dp[0]=0;cnt[0]=0;
    hd=tl=1;q[1]={0,1};
    for(int i=1;i<=n;i++){
        while(hd<tl&&q[hd+1].l<=i)++hd;
        take(q[hd].j,i,extra);
        insert_candidate(i);
    }
}
```

和 WQS 合用时，令 `extra=k`，一次 `check(k)` 就是一次 $O(n \log n)$ 的上述 DP，于是总复杂度再乘一个 WQS 的二分次数。

多测与注意 队列与 dp[0] 每次已重置，cost 依赖的数据按组建。含 tie-break 的候选比较需单次切换，复杂度还取决于 cost。

4 字符串 (String Algorithms)

4.1 字符串哈希 (String Hashing)

用于 O(n) 预处理，O(1) 查询任意子串的哈希值。这个板子好像挺容易冲突的，建议双 hash。

```
const ll mod=180181327,bs=31;
ll pw[1000010];
ll h[1000010],n;
char c[1000010];
ll get(ll l,ll r){
    return (h[r]-pw[r-l+1]*h[l-1]%mod+mod)%mod;
}
int main(){
    pw[0]=1;f(i,1,1000000)pw[i]=pw[i-1]*bs%mod;
    scanf("%s",c+1);n=strlen(c+1);h[0]=0;
    f(i,1,n)h[i]=(h[i-1]*bs+(c[i]-'a'+1))%mod;
}
```

多测与注意 每组重建 h，固定参数的幂表可预处理一次。保留原单模数写法，实战建议双 hash。

4.2 KMP

Knuth-Morris-Pratt 算法，用于在线性时间内进行字符串匹配（在 c 串中找 s 串）。

```
char c[1000010],s[1000010];
ll nx[1000010];
int main(){
  cin>>c+1>>s+1;
  n=strlen(c+1),m=strlen(s+1);
  nx[1]=0;ll j=0;f(i,2,m){
    while(j&&s[j+1]!=s[i])j=nx[j];
    if(s[j+1]==s[i])j++;nx[i]=j;
  }j=0;f(i,1,n){
    while(j&&s[j+1]!=c[i])j=nx[j];
    if(s[j+1]==c[i])j++;
    if(j==m){cout<<i-m+1<<'\n';j=nx[j];}
  }
}
```

多测与注意 每组 nx[1]=0、j=0，其他有效区间覆盖即可；模式串非空。原 C++17 字符数组输入保留。

4.3 Manacher

Manacher 算法，用于在线性时间内求解字符串的最长回文子串。

```
  m=strlen(C+1);n=0;
  c[++n]='#';
  f(i,1,m)c[++n]=C[i],c[++n]='#';
  f(i,1,n)res[i]=0;
    md=1,r=1;ans=0;
  f(i,2,n){
    if(i>r){
      while(i+res[i]<n&&i-res[i]>1&&c[i+res[i]+1]==c[i-res[i]-1])res[i]++;
      md=i,r=i+res[i];
    }else{
      res[i]=min(r-i,res[md*2-i]);
      while(i+res[i]<n&&i-res[i]>1&&c[i+res[i]+1]==c[i-res[i]-1])res[i]++;
      if(i+res[i]>r)r=i+res[i],md=i;
    }ans=max(ans,res[i]);
  }
  cout<<ans;
```

多测与注意 保留原插井号写法，只补 n、res、ans 的逐组重置。处理串数组开到原串约两倍。

4.4 AC 自动机 (Aho-Corasick)

字符集过大时，暴力跳 fail 是对的。

```
ll ch[1000010][26];
ll fail[1000010];
vi e[1000010];
void add(ll nd){ll u=1;
  ll len=strlen(c+1);
  f(i,1,len){
    ll p=c[i]-'a';
    if(!ch[u][p])ch[u][p]=++cnt;
```

```
    u=ch[u][p];
  }
}queue<ll>q;
void gtf(){
  fail[1]=1;
  f(i,0,25){
    if(ch[1][i])q.push(ch[1][i]),fail[ch[1][i]]=1;
    else ch[1][i]=1;
  }while(!q.empty()){
    ll u=q.front();q.pop();
    f(i,0,25){
      if(ch[u][i])fail[ch[u][i]]=ch[fail[u]][i],q.push(ch[u][i]);
      else ch[u][i]=ch[fail[u]][i];
    }
  }
}

//字符集过大时，暴力跳 fail 是对的。用 find(u,i) 代替 ch[u][i]（getfail里和跑acam的时候）
ll find(ll u,ll i){
  while(u!=1&&!ch[u][i])u=fail[u];
    return ch[u][i]?ch[u][i]:1;
}
void init(){
    f(i,0,cnt){fill(ch[i],ch[i]+26,0);fail[i]=0;e[i].clear();}
    cnt=1;while(!q.empty())q.pop();
}
```

多测与注意 每组先 init，再全部 add、gtf；保留根编号 1。gtf 补全转移后不要再插模式或重复构建。稀疏版 find 只用于未补全的 trie 转移。

4.5 后缀数组 (Suffix Array - SA)

he: 排名 i 和排名 i-1 的后缀的 LCP

```
ll n,m;
char c[1000010];
ll sa[1000010],rk[1000010];
struct node{ll x,y,id;}A[1000010],B[1000010];
ll he[1000010];
ll buc[1000010];
void SA(){
    fill(buc,buc+max(n,256LL)+1,0);he[1]=0;
  f(i,1,n)buc[(unsigned char)c[i]]++;
  f(i,1,256)buc[i]+=buc[i-1];
  f(i,1,n)rk[i]=buc[(unsigned char)c[i]-1]+1;
  for(int l=1;l<=n;l*=2){
    f(i,1,n){
      if(i+l>n)A[i]={rk[i],0,i};
      else A[i]={rk[i],rk[i+l],i};
    }f(i,0,n)buc[i]=0;f(i,1,n)buc[A[i].y]++;
    f(i,1,n)buc[i]+=buc[i-1];
    for(int i=n;i>=1;i--)B[buc[A[i].y]--]=A[i];
    f(i,0,n)buc[i]=0;f(i,1,n)buc[B[i].x]++;
    f(i,1,n)buc[i]+=buc[i-1];
    for(int i=n;i>=1;i--)A[buc[B[i].x]--]=B[i];
    ll cur=1;rk[A[1].id]=1;
    f(i,2,n){
      if(A[i].x==A[i-1].x&&A[i].y==A[i-1].y);
```

```
      else cur=i;rk[A[i].id]=cur;
    }
  }f(i,1,n)sa[rk[i]]=i;
  ll k=0;f(i,1,n){
    if(k)k--;if(rk[i]==1){k=0;continue;}
    ll j=sa[rk[i]-1];
    while(i+k<=n&&j+k<=n&&c[i+k]==c[j+k])k++;
    he[rk[i]]=k;
  }
}//he:排名 i 和排名 i-1 的后缀的LCP
```

多测与注意 恢复原数组版、1 下标、倍增计数排序。已补初始桶及 0 号桶清理，SA 每组可重新调用；字符串按原非空输入使用。

4.6 回文自动机 (Palindrome Automaton - PAM)

```
char C[500010];
char c;
ll res;
ll ch[500010][26];
ll fail[500010];
ll len[500010];
ll sz[500010];
ll lst;
void add(ll u){
  while(1){
    if(c==C[n-len[u]-1]){
      if(!ch[u][c-'a'])ch[u][c-'a']=++cnt,len[cnt]=len[u]+2;
      lst=ch[u][c-'a'];
      break;
    }u=fail[u];
  }if(u==1){sz[ch[u][c-'a']]=1,fail[ch[u][c-'a']]=0;return;}
  u=fail[u];
  while(1){
    if(c==C[n-len[u]-1]){
      fail[lst]=ch[u][c-'a'];
      sz[lst]=sz[ch[u][c-'a']]+1;
      break;
    }u=fail[u];
  }
}char fake[500010];
int main(){
  f(i,0,cnt){fill(ch[i],ch[i]+26,0);fail[i]=len[i]=sz[i]=0;}
    n=res=0;C[0]=0;
    len[0]=0,len[1]=-1;
  cnt=1;fail[0]=1;fail[1]=-1;lst=1;
  scanf("%s",fake+1);m=strlen(fake+1);
  f(i,1,m){
    c=(fake[i]-97+res)%26+97;
    C[++n]=c;add(lst);
    res=sz[lst];
    printf("%lld ",res=sz[lst]);//以这个位置结尾的回文子串个数
  }return 0;
}
```

多测与注意　保留原加密读入、add、sz 和根编号，只补逐组清空。若题目无字符加密，把计算 c 的那行改为 c=fake[i]。

5 数学 (Mathematics)

5.1 数论 (Number Theory)

5.1.1 Exgcd (Extended Euclidean Algorithm)

扩展欧几里得算法，用于求解形如 $ax + by = c$ 的不定方程。

```
void exgcd(ll a,ll b){
  if(b==0){x=c/a;y=0;return;}
  exgcd(b,a%b);ll xx=x,yy=y;
  x=yy,y=xx-a/b*yy;
}
```

多测与注意 保留原求 $ax + by = c$ 的写法。调用前先判断 gcd(a,b) 是否整除 c；x、y 每次会重算，a=b=0 按题意特判。

5.1.2 万能欧几里得

把 linear 看成 U（穿过横轴）和 R（穿过纵轴）的操作序列，复合操作来优化复杂度。

```
//Operation.
//Cross a horizontal line => U operation (impact of floor sum+1)
//Cross a vertical line => R operation (impact of i+1)
struct node{
    ll s,numU,numR;
    node(ll s=0,ll numU=0,ll numR=0){
        this->s=s,this->numU=numU,this->numR=numR;
    }
    node operator + (const node& b){
        node c;
        c.numR=numR+b.numR;
        c.numU=numU+b.numU;
        c.s=s+b.s+numU*b.numR;
        return c;
    }
};
//pow of operation
node opow(node a,ll b){
    node res;
    while(b){
        if(b&1)res=res+a;
        b>>=1,a=a+a;
    }return res;
}
// need a>=0, b>=0, b<m
// 计算 [1,n] 的UR复合操作的结果
node work(ll a,ll b,ll n,ll m,node U,node R){
    if(!n)return node();//构造函数应该取单位元，若没有单位元则需要小心处理opow里b=0和本行的情况。
    if(a>=m)return work(a%m,b,n,m,U,opow(U,a/m)+R);
    ll cu=(ll)(((__int128)n*a+b)/m);//Exact integer division; widen node fields too if needed.
    if(!cu)return opow(R,n);
    ll cnt=n-(ll)(((__int128)m*cu-b-1)/a);//Exact integer division; widen node fields too if needed.
    return opow(R,(m-b-1)/a)+U+work(m,(m-b-1)%a,cu-1,a,R,U)+opow(R,cnt);
}
node U,R;

// 使用例: \sum_{i=0}^{n-1} \lfloor\frac{ai+b}{m}\rfloor
```

```
// for b>=m, use opow(U,b/m)+work(a,b%m,n,m,U,R) to obtain the result.
void solve(){
  U=(node){0,1,0},R=(node){0,0,1};
    n=d-1,m=d;ll a=d,b=d;
    node res=opow(U,b/m)+work(a,b%m,n,m,U,R);//b>=m的时候要线提出几个 U 来用。
    printf("%lld\n",res.s+b/m);
    // out(res.s+b/m);puts("");
}
```

多测与注意 U、R 每组重建；work 要求 $a \geq 0, 0 \leq b < m$。只将整除中间积改 128 位；若结果本身大，node 累加量也要换类型。空求和在 solve 前特判。

5.1.3 ExCRT (Extended Chinese Remainder Theorem)

扩展中国剩余定理，用于求解模数不互质的线性同余方程组。

```
ll x,y,c;
void exgcd(ll a,ll b){
  if(b==0){x=1,y=0;return;}
  exgcd(b,a%b);ll xx=x,yy=y;
  x=yy,y=xx-a/b*yy;
}ll gcd(ll a,ll b){
  return (b==0)?a:gcd(b,a%b);
}
ll A,B,a,b;
// merge {x equiv A mod B} and {x equiv a mod b}
bool merge(ll &A,ll &B,ll a,ll b){
    c=a-A;ll g=gcd(b,B);
    if(c%g)return false;
    exgcd(B,b);
    x=(__int128)x*(c/g)%(b/g);b=b/g*B;
    A=((A+(__int128)x*B%b)%b+b)%b;B=b;return true;
}
```

多测与注意 保留 A、B、x、y、c 及原合并流程，merge 仅增加 bool 返回值表示有无解。每组 A=0、B=1 开始；若周期超 ll，按题目改类型。

5.1.4 扩展欧拉定理 (Euler's Totient Theorem)

扩展欧拉定理，用于处理 b 很大的幂取模。当 $a, m \in \mathbb{Z}$ 时有：

$$a^b \equiv \begin{cases} a^b, & b < \varphi(m) \\ a^{b \bmod \varphi(m) + \varphi(m)}, & b \geq \varphi(m) \end{cases} \pmod{m}$$

多测与注意 幂塔需同时记指数是否达到 phi，不能只记余数。mod=1 结果为 0。

5.1.5 Pollard-Rho

Pollard's Rho 算法，一种用于大整数质因数分解的随机算法。

```
ll qp(ll a,ll b,ll p){a%=p;
  ll ans=1;while(b){
    if(b&1)ans=(__int128)ans*a%p;
    a=(__int128)a*a%p;b>>=1;
```

```
  }return ans;
}ll n,m;
ll tst[7]={2,325,9375,28178,450775,9780504,1795265022};
/*
unsigned : 2 7 61
ull : 2 325 9375 28178 450775 9780504 1795265022
*/
bool miller(ll x){if(x<2)return 0;
    if(x==2||x==3||x==5||x==7||x==11||x==13)return 1;
  ll u=x-1,t=0;while(u%2==0)u/=2,t++;
  f(T,0,6){
    ll a=tst[T];
    if(a%x==0)continue;
    ll b=qp(a,u,x),bb;//if(b==1||b==x-1)continue;
    f(i,1,t){
      bb=(__int128)b*b%x;
      if(bb==1&&b!=1&&b!=x-1)return 0;b=bb;
    }if(b!=1)return 0;
  }return 1;
}ll pollard(ll p){
  ll c=(ll)rand()%p+1;
  ll s=0,t=0,mul=1,k=1,i=0;
  for(k=1;;k<<=1,s=t,mul=1){
    for(i=1;i<=k;i++){
      t=((__int128)t*t+c)%p;
      mul=(__int128)mul*abs(s-t)%p;
      if(i%127==0){
        ll g=__gcd(mul,p);
        if(g>1)return g;
      }
    }ll g=__gcd(mul,p);
    if(g>1)return g;
  }
}ll mxp;
void work(ll p){
  if(p<2||p<=mxp)return;
  if(miller(p)){mxp=max(mxp,p);return;}
  ll k=p;while(k==p)k=pollard(p);
  while(p%k==0)p/=k;
  work(p),work(k);
}
int main(){srand(time(0));
  wt{ ll p=d;mxp=0;
    work(p);if(mxp==p)puts("Prime");
    else printf("%lld\n",mxp);
  }
  return 0;
}
```

多测与注意　保留原 rand 和最大质因子递归。每个输入 mxp=0 已有；底数为 n 倍数时已改为跳过。n=1 按题意处理。

5.1.6 Dirichlet 前缀和 (Dirichlet Prefix Sum)

Dirichlet 前缀和，用于快速计算形如 $f(i)=\sum_{x|i} a_x$ 的函数。

$$a_i \to \sum_{x|i} a_x$$

```
for(i=1;i<=cnt;++i)for(j=1;j*pr[i]<=n;++j)a[j*pr[i]]+=a[j];
```

多测与注意 素数表可复用；每组 a 恢复原输入。j 递增是约数和变换，逆变换反序做减法。

5.1.7 杜教筛 (Du's Sieve)

杜教筛，用于在亚线性时间内求解数论函数（如 μ, ϕ）的前缀和。设 $S(n) = \sum_{i=1}^{n} f(i)$，我们构造 g 使得 $\sum(f * g)$ 和 $\sum g$ 易于计算。其核心公式为：

$$g(1)S(n) = \sum_{i=1}^{n}(f * g)(i) - \sum_{i=2}^{n} g(i)S(\lfloor n/i \rfloor)$$

```
ll n,m;
ll pr[2000010];
bool vis[2000010];
ll mu[2000010];
ll ph[2000010];
void gt(){
    static bool done=0;if(done)return;done=1;cnt=0;
  mu[1]=ph[1]=1;
  f(i,2,2000000){
    if(!vis[i])pr[++cnt]=i,mu[i]=-1,ph[i]=i-1;
    for(int j=1;j<=cnt&&i*pr[j]<=2000000;j++){
      vis[i*pr[j]]=1;if(i%pr[j]==0){
        ph[i*pr[j]]=ph[i]*pr[j];break;
      }mu[i*pr[j]]=-mu[i],ph[i*pr[j]]=ph[i]*ph[pr[j]];
    }
  }f(i,2,2000000)mu[i]+=mu[i-1],ph[i]+=ph[i-1];
}
unordered_map<ll,ll>smu;
unordered_map<ll,ll>sph;
ll Mu(ll n){
  if(n<=2000000)return mu[n];
  if(auto it=smu.find(n);it!=smu.end())return it->second;
  ll res=1;for(ll l=2,r;l<=n;l=r+1){
    r=n/(n/l);res-=(r-l+1)*Mu(n/l);
  }return smu[n]=res;
}ll Ph(ll n){
  if(n<=2000000)return ph[n];
  if(auto it=sph.find(n);it!=sph.end())return it->second;
  ll res=n*(n+1)/2;for(ll l=2,r;l<=n;l=r+1){
    r=n/(n/l);res-=(r-l+1)*Ph(n/l);
  }return sph[n]=res;
}
int main(){
  gt();wt{
    n=d;printf("%lld %lld\n",Ph(n),Mu(n));
  }
  return 0;
}
```

多测与注意 gt 一次，前缀表和 memo 可跨组保留。只修正“缓存值为 0 时重复计算”；筛上限按题目调整。

5.1.8 Min25

解决的问题：求积性函数 f 的 $\sum_{i=1}^{n} f(i)$。亚线性，一般 1s 能跑 $n \le 10^{11}$。

定义：

$$S(m,i) = \sum_{2\le j\le m,\min_p(j)>p_i} f(j)$$

那么有递推：

$$S(m,i) = \sum_{k=i+1}^{p_k\le m} f(p_k) + \sum_{k>i}\sum_{e=1}^{p_k^e\le m} f(p_k^e)\left([e>1] + S(\lfloor\frac{m}{p_k^e}\rfloor, k)\right)$$

其中最后的答案就是 $S(n,0)+f(1) = S(n,0)+1$。

递推瓶颈在于 $\sum_{k=i+1}^{p_k\le m} f(p_k) = \sum_{k=1}^{p_k\le m} f(p_k) - \sum_{k=1}^{i} f(p_k)$。后者上界不会超过 $\pi(\sqrt{n})$，可以线性筛预处理，前者（即质数处点值和）处理方法如下：

定义 $g_c(m,i)$ 是一个完全积性函数 f_c 的全局前缀和（$\sum_{i=2}^{m} f_c(i)$）去掉 $min_p(j)\le p_i$ 的合数的点值的和。即：

$$g_c(m,i) = \sum_{k=i+1}^{p_k\le m} f_c(p_k) + \sum_{2\le j\le m,\min_p(j)>p_i} f_c(j)$$

有递推式：

$$g_c(m,i) = g_c(m,i-1) - f_c(p_i)\left(g_c(\lfloor\frac{m}{p_i}\rfloor, i-1) - \sum_{j=1}^{i-1} f_c(p_j)\right)$$

可以类似 dp 方式递推。

最后 $g_c(m,\pi(\sqrt{m}))$ 即为质数处点值和。

对于非完全积性函数 f，我们如果能把 f 写成若干个完全积性函数的线性组合 $f(n) = \sum a_i f_i$，则其前缀和可以写成 $\sum a_i(S_i(n,0)+1)$，即可使用 Min25 筛处理。

```
const ll mod=1000000007;
ll qp(ll a,ll b=mod-2){
    ll res=1;res%=mod;
    while(b){
        if(b&1)res=res*a%mod;
        a=a*a%mod,b>>=1;
    }return res;
}
ll pr[100010],cnt;
bool vis[100010];
const ll iv2=qp(2),iv6=qp(6);
ll sum1(ll n){
    n%=mod;
    return n*(n+1)%mod*iv2%mod;
}ll sum2(ll n){
    n%=mod;
    return n*(n+1)%mod*(2*n+1)%mod*iv6%mod;
}
ll s1[300010];// sum p
ll s2[300010];// sum p^2
void gt(ll lim){
    cnt=0;fill(vis,vis+lim+1,0);s1[0]=s2[0]=0;
```

```
    f(i,2,lim){
        if(!vis[i])pr[++cnt]=i;
        for(int j=1;j<=cnt&&i*pr[j]<=lim;j++){
            vis[i*pr[j]]=1;
            if(i%pr[j]==0)break;
        }
    }f(i,1,cnt){
        s1[i]=(s1[i-1]+pr[i])%mod;
        s2[i]=(s2[i-1]+pr[i]*pr[i])%mod;
    }//预处理完全积性函数前缀和
}ll n,tot;
ll a[300010];
ll g0[300010];//f(i)=1
ll g1[300010];//f(i)=i
ll g2[300010];//f(i)=i^2
int id1[400010],id2[400010];
int lim;
ll S(ll x,ll id){
    if(pr[id]>a[x])return 0;
    ll res=(g2[x]-g1[x]-(s2[id]-s1[id])+2*mod)%mod;// 对应质数处点值前缀和的差分
    f(i,id+1,cnt)if(pr[i]*pr[i]<=a[x]){
        ll t=pr[i],o=1;
        while(1){
            int iid=(a[x]/t>lim)?id2[n/(a[x]/t)]:id1[a[x]/t];
            res=(res+t%mod*(t%mod-1+mod)%mod*(S(iid,i)+(o>1)))%mod;//此处对应 S 的递推，(t%mod*(t%
    mod-1+mod)%mod) 是 f(p^e)
            if(t>a[x]/pr[i])break;t*=pr[i];o++;
        }
    }else break;
    return res;
}
int main(){
  n=d;tot=0;gt(sqrt(n));lim=sqrt(n);
    for(ll l=1,r;l<=n;l=r+1){
        r=n/(n/l);tot++;
        if(n/l<=lim)id1[n/l]=tot;
        else id2[r]=tot;
        a[tot]=n/l;
        // g0[tot]=(n/l-1)%mod;
        g1[tot]=sum1(n/l)-1;if(g1[tot]<0)g1[tot]+=mod;
        g2[tot]=sum2(n/l)-1;if(g2[tot]<0)g2[tot]+=mod;
    }
    f(i,1,cnt){
        f(j,1,tot)if(pr[i]*pr[i]<=a[j]){
            // g0[j]=(g0[j]+mod-(g0[mp[a[j]/pr[i]]]-(i-1)+mod)%mod)%mod;
            int id=(a[j]/pr[i]>lim)?id2[n/(a[j]/pr[i])]:id1[a[j]/pr[i]];
            g1[j]=(g1[j]+mod-pr[i]*(g1[id]-s1[i-1]+mod)%mod)%mod;
            g2[j]=(g2[j]+mod-pr[i]*pr[i]%mod*(g2[id]-s2[i-1]+mod)%mod)%mod;
        }else break;
    }
    cout<<(S(1,0)+1)%mod;
  return 0;
}
```

多测与注意　恢复原静态数组和商块编号，补 cnt、vis、tot 的逐组重置。数组按 $\sqrt{n}$ 与商块数开。此例素数幂点值为 $p^e(p^e-1)$，换函数需改对应部分；原文关于一般函数的泛化不能直接当作证明。

5.1.9 单位根反演

就一个式子。

$$[n \mid m] = \frac{1}{n}\sum_{i=0}^{n-1}\omega_n^{im}$$

还有一个进阶式子。

$$[m \bmod n = k] = [n|(m-k)] = \frac{1}{n}\sum_{i=0}^{n-1}\omega_n^{im}\omega_n^{-ik}$$

模意义下 $\omega_n = g^{\frac{p-1}{n}}$

多测与注意 模质数 p，要求 $n \mid (p-1)$ 且 g 为原根；换 p 或 n 重算根。

5.2 组合数学 (Combinatorics)

5.2.1 组合数 (Binomial Coefficients)

预处理阶乘及其逆元，O(1) 查询组合数。

```
namespace binom{
  const ll Lim=300010,mod=998244353;
  ll jc[Lim],inv[Lim],inc[Lim];
  void pre(){
    jc[0]=jc[1]=inc[0]=inc[1]=inv[0]=inv[1]=1;
    f(i,2,Lim-1)jc[i]=jc[i-1]*i%mod,inv[i]=(mod-mod/i)*inv[mod%i]%mod,
    inc[i]=inc[i-1]*inv[i]%mod;
  }ll C(ll n,ll m){if(n<0||m<0||n<m)return 0;return jc[n]*inc[m]%mod*inc[n-m]%mod;}
}
```

多测与注意 同模数 pre 一次即可，表按题目开；阶乘预处理范围要小于素数 mod。

5.2.2 Lucas 定理 (Lucas's Theorem)

Lucas 定理，用于在 O(p $\log_p n$) 时间内求解模小质数 p 的大组合数。（除法为下取整，要求 p 为质数）

$$\binom{n}{m} \equiv \binom{n \bmod p}{m \bmod p}\binom{\lfloor n/p \rfloor}{\lfloor m/p \rfloor} \pmod p$$

多测与注意 同 p 可复用阶乘表；$O(p)$ 预处理后，单次 Lucas 为 $O(\log_p n)$。

5.2.3 ExLucas (Extended Lucas Theorem)

扩展 Lucas 定理，用于求解模任意数 P 的大组合数。

```
ll n,m,P;
ll pr[1000010],cnt;
bool vis[1000010];
void gt(){
    static bool done=0;if(done)return;done=1;cnt=0;
  f(i,2,1000000){
    if(!vis[i])pr[++cnt]=i;
    for(int j=1;j<=cnt&&i*pr[j]<=1000000;j++){
```

```
      vis[i*pr[j]]=1;if(i%pr[j]==0)break;
    }
  }
}
ll eX,eY,eC;
void exgcd(ll a,ll b){
  if(b==0){eX=eC/a,eY=0;return;}
  exgcd(b,a%b);ll x=eX,y=eY;
  eX=y,eY=x-a/b*y;
}ll inv(ll x,ll p){eC=1;exgcd(x,p);return (eX%p+p)%p;}
ll F(ll n,ll p,ll pk){
  if(n==0)return 1;
  ll res=F(n/p,p,pk);
  ll g=1;f(i,1,pk)if(i%p)g=g*i%pk;
  g=qp(g,n/pk,pk);f(i,1,n%pk)if(i%p)g=g*i%pk;
  return res*g%pk;
}ll c(ll n,ll p){
  ll res=0;while(n)res+=n/p,n/=p;
  return res;
}ll C(ll n,ll m,ll P){
  if(n<m||m<0||P==1)return 0;
  ll A=-1,B=-1,nn=P;
  f(i,1,cnt+1){
        ll p=(i<=cnt?pr[i]:nn);if(p<=1||nn%p)continue;
        ll pk=1;
    while(nn%p==0)nn/=p,pk*=p;
    ll AA=F(n,p,pk)*inv(F(m,p,pk),pk)%pk*inv(F(n-m,p,pk),pk)%pk;
    AA=AA*(qp(p,c(n,p)-c(m,p)-c(n-m,p),pk))%pk;ll BB=pk;
    if(A==-1)A=AA,B=BB;
    else {
      if(A<AA)swap(A,AA),swap(B,BB);eC=A-AA;
      exgcd(B,BB);eX=((-eX)%BB+BB)%BB;
      BB*=B;A=(eX*B+A)%BB;B=BB;
    }
  }return A;
}
int main(){
  gt();n=d,m=d,P=d;
  printf("%lld",C(n,m,P));
  return 0;
}
```

多测与注意　保留原 F、C、exgcd 和 CRT 合并，补 P=1 与残余质因子。gt 一次；素数表开到所需试除范围。F 仍逐层扫素数幂，模数大时耗时高，可按题目缓存乘积块，未改写实现。

5.2.4　min-max 容斥

$$\text{k-th}\max(S)=\sum_{\emptyset\neq T\subseteq S,|T|\geq k}(-1)^{|T|-k}\binom{|T|-1}{k-1}\min(T)$$

$$\text{k-th}\min(S)=\sum_{\emptyset\neq T\subseteq S,|T|\geq k}(-1)^{|T|-k}\binom{|T|-1}{k-1}\max(T)$$

多测与注意　$1\leq k\leq|S|$，重复值按位置计子集。公式本身无多测状态。

5.2.5 反演

1. 二项式反演

$$f(n)=\sum_{i=0}^{n}\binom{n}{i}g(i)\iff g(n)=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f(i)$$

$$f(k)=\sum_{i=k}^{n}\binom{i}{k}g(i)\iff g(k)=\sum_{i=k}^{n}(-1)^{i-k}\binom{i}{k}f(i)$$

2. 子集反演

$$f(S)=\sum_{T\subseteq S}g(T)\iff g(S)=\sum_{T\subseteq S}(-1)^{|S|-|T|}f(T)$$

3. 斯特林反演

$$f(n)=\sum_{k=0}^{n}\begin{Bmatrix}n\\k\end{Bmatrix}g(k)\iff g(n)=\sum_{k=0}^{n}(-1)^{n-k}\begin{bmatrix}n\\k\end{bmatrix}f(k)$$

$$f(k)=\sum_{i=k}^{n}\begin{Bmatrix}i\\k\end{Bmatrix}g(i)\iff g(k)=\sum_{i=k}^{n}(-1)^{i-k}\begin{bmatrix}i\\k\end{bmatrix}f(i)$$

其中，

$$\begin{bmatrix}n\\k\end{bmatrix}=(n-1)\begin{bmatrix}n-1\\k\end{bmatrix}+\begin{bmatrix}n-1\\k-1\end{bmatrix}$$

$$\begin{Bmatrix}n\\k\end{Bmatrix}=k\begin{Bmatrix}n-1\\k\end{Bmatrix}+\begin{Bmatrix}n-1\\k-1\end{Bmatrix}$$

边界条件：$S(n,n)=1,S(n,0)=0(n>0)$，两类都是。

多测与注意 两类都有 S(0,0)=1；第一类为无符号版本。预处理可复用，滚动数组清目标行。

5.3 线性代数 (Linear Algebra)

5.3.1 高斯消元 (Gaussian Elimination)

高斯消元法，用于求解线性方程组。

```
double a[110][110];
double b[110];
ll p[110];
const double eps=1e-7;
int main(){
  n=d;f(i,1,n){
    f(j,1,n+1)cin>>a[i][j];
  }ll row=1;
    f(i,1,n){
    ll pos=row;f(j,row,n)if(fabs(a[j][i])>fabs(a[pos][i]))pos=j;
        if(row>n||fabs(a[pos][i])<eps)continue;
    swap(a[row],a[pos]);
    double x=a[row][i];f(j,1,n+1)a[row][j]/=x;
    f(j,row+1,n){
```

```
      x=a[j][i];
      f(k,i,n+1)a[j][k]-=x*a[row][k];
    }row++;
  }
    // f(i,1,n){
    //     f(j,1,n+1)cout<<a[i][j]<<" ";cout<<endl;
    // }
    f(i,1,n){
        bool fl=1;
        f(j,1,n)if(fabs(a[i][j])>eps)fl=0;
        if(fl){if(fabs(a[i][n+1])>eps){puts("-1");return 0;}}//无解
    }
    f(i,1,n){
        bool fl=1;
        f(j,1,n)if(fabs(a[i][j])>eps)fl=0;
        if(fl){if(fabs(a[i][n+1])<=eps){puts("0");return 0;}}//无限解
    }
    for(int i=n;i>=1;i--){
    b[i]=a[i][n+1];
    f(j,1,i-1)a[j][n+1]-=a[i][n+1]*a[j][i];
  }
    f(i,1,n)printf("x%lld=%.2lf\n",(long long)i,b[i]);
  return 0;
}
```

多测与注意 仍是原矩阵和消元流程，只选最大绝对主元并支持实数输入。每组重读矩阵，row=1；消元破坏原矩阵，eps 按题意取。

5.3.2 行列式

如果模数不是质数，需要对行的主元进行辗转相除。

```
ll qp(ll a,ll b){
    ll res=1;
    while(b){
        if(b&1)res=res*a%mod;
        a=a*a%mod,b>>=1;
    }return res;
}
ll det(ll n,ll a[][1010]){
    int sgn=1;
    f(i,1,n)f(j,1,n)a[i][j]=(a[i][j]%mod+mod)%mod;
    f(i,1,n){
        int p=-1;
        f(j,i,n)if(a[j][i]!=0){p=j;break;}
        if(p==-1){return 0;}
        if(p!=i)swap(a[i],a[p]),sgn=-sgn;
        ll iv=qp(a[i][i],mod-2);
        f(j,i+1,n){
            ll o=a[j][i]*iv%mod;
            f(k,i,n)a[j][k]=(a[j][k]-a[i][k]*o%mod+mod)%mod;
        }
    }
    ll res=1;
    f(i,1,n)res=res*a[i][i]%mod;
    if(sgn==-1)res=(mod-res)%mod;
    return res;
}
```

多测与注意 本实现用 Fermat 逆元，要求素数 mod；合数模换辗转消元。det 原地改矩阵，每组重建。

5.3.3 矩阵树定理

General Form: The sum of weights of all spanning trees is given by the cofactor of the Laplacian matrix:

$$\sum_{T\in\mathcal{T}}\prod_{e\in T} w_e = \det(L^*)$$

1. Undirected Graph

$$L_{ij} = \begin{cases} \deg(i) & i = j \\ -w_{ij} & i \neq j \end{cases}$$

Result: $\det(L)$ after removing **any** k-th row and column.

2. Directed Graph (Root r) Remove the r-th row and column from L to calculate determinant.

A. Out-Tree ($r \to$ all): Use **In-Degree** & A^T.

$$L_{ij} = \begin{cases} \deg^{in}(i) & i = j \\ -w_{ji} & i \neq j \quad (\text{edge } j \to i) \end{cases}$$

B. In-Tree (all $\to r$): Use **Out-Degree** & A.

$$L_{ij} = \begin{cases} \deg^{out}(i) & i = j \\ -w_{ij} & i \neq j \quad (\text{edge } i \to j) \end{cases}$$

当然，此处的 w 可以替换为任意有交换律的东西，比如形式幂级数。

```
void solve(){
    n=d,m=d;f(i,1,n)f(j,1,n)a[i][j]=0;ll o=d;//0 无向; 1 有向且 1 为根外向树
    if(o==0){
        f(i,1,m){
            ll u=d,v=d,w=d;
            a[u][u]+=w,a[v][v]+=w;
            a[u][v]-=w,a[v][u]-=w;
        }
        f(i,2,n)f(j,2,n)a[i-1][j-1]=a[i][j];
        printf("%lld\n",det(n-1,a));
    }if(o==1){
        f(i,1,m){
            ll u=d,v=d,w=d;
            a[v][v]+=w;
            a[v][u]-=w;
        }
        f(i,2,n)f(j,2,n)a[i-1][j-1]=a[i][j];
        printf("%lld\n",det(n-1,a));
    }
}
```

多测与注意 已补清本组矩阵。平行边权相加、自环抵消；换根多次求值先存原矩阵。调用上方 det 仍需素数模。

5.3.4 BEST 定理 (BEST Theorem)

BEST 定理，用于求解**有向欧拉图**的欧拉回路计数。设 $G=(V,E)$ 是有向欧拉图（入度等于出度且连通），t_{root} 为以任意点 $root$ 为根的**根向树形图**（Arborescence）个数，则 G 的欧拉回路总数 $ec(G)$ 为：

$$ec(G)=t_{root}\cdot\prod_{v\in V}(\deg(v)-1)!$$

注：t_{root} 通常配合**矩阵树定理**（Matrix Tree Theorem）求解。

多测与注意 公式为边可区分、固定首边的计数；仅固定起点再乘根的出度。阶乘忽略零度孤立点，每组重建度数和矩阵。

5.3.5 LGV 引理 (Weighted LGV Lemma)

LGV 引理，适用于 **DAG** 上带权不相交路径问题。设边权为 $w(e)$（可为数值或多项式），定义路径 P 的权值为 $\omega(P)=\prod_{e\in P}w(e)$。构造矩阵 M，其中 M_{ij} 为起点 a_i 到终点 b_j 的所有路径权值之和：

$$M_{ij}=\sum_{P:a_i\to b_j}\omega(P)=\sum_{P:a_i\to b_j}\prod_{e\in P}w(e)$$

若图满足条件（如平面图且起点终点排序使得仅 $\sigma=id$ 存在不相交路径），则所有不相交路径组 $(P_1,\dots,P_n)$ 的权值积之和为：

$$\det(M)=\sum_{(P_1,\dots,P_n)\in\text{Disjoint}}\prod_{k=1}^{n}\omega(P_k)$$

注：若求不相交路径总数，设 $w(e)=1$；若求路径权值总和，设 $w(e)=x^{val}$ 并做多项式运算。

多测与注意 一般行列式给出带排列符号的点不交路径族和；要作无符号计数，需证明只有恒等配对可行。每组重建路径 DP 和矩阵。

5.4 多项式 (Polynomials)

多项式全家桶，包含 NTT, Inv, Ln, Exp, Pow, Div 等操作。

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define f(i,a,b) for(int i=a;i<=b;++i)
#define wt int tt=d;while(tt--)
#define py puts("Yes")
#define pn puts("No")
#define fe(i,e) for(int i=0;i<e.size();i++)
#define vi vector<ll>
inline ll rd() {
  ll x=0,f=1;
  char c=getchar();
  while(!isdigit(c)){if(c=='-')f=-1;c=getchar();}
  while(isdigit(c))x=x*10+c-'0',c=getchar();
  return x*f;
```

```
}
ll dx[4]={0,1,0,-1};
ll dy[4]={1,0,-1,0};
#define d rd()
#define pb push_back
const ll Bit=21;
const ll PoL=(1<<Bit)+5;
ll but[PoL],pw[2][PoL],Curl=-1;
void out(ll *a,ll n){f(i,0,n)printf("%lld ",a[i]);puts("");}
const ll mod=998244353,G=3,Gi=332748118;
ll qp(ll a,ll b=mod-2){ll ans=1;
  while(b){
    if(b&1)ans=ans*a%mod;
    a=a*a%mod;b>>=1;
  }return ans%mod;
}
namespace poly{
  ll jc[PoL+10],inv[PoL+10],inc[PoL+10];
  ll w[PoL]={1},qwq=0,Curl=-1;
  // ll usla[PoL],uslb[PoL];
  bool hpre=0;
  void pre(){
    if(hpre)return;hpre=1;
    jc[0]=jc[1]=inc[0]=inc[1]=inv[0]=inv[1]=1;
    f(i,2,PoL-1)jc[i]=jc[i-1]*i%mod,inv[i]=(mod-mod/i)*inv[mod%i]%mod,
    inc[i]=inc[i-1]*inv[i]%mod;
  }
  #define ck(x) ((x)>=mod?(x)-mod:(x))
  void revbit(ll k){
    if(Curl==k)return;Curl=k;
    f(i,0,k-1)but[i]=but[i>>1]>>1|((i&1)?(k>>1):0);
  }
  ll NTT(ll *a,ll *to,ll n,ll o){
    ll _n=1;while(_n<=n)_n<<=1;n=_n;
    revbit(n);qwq=0;
    ll inv=qp(n);

    // memcpy(to,a,sizeof(to));
    f(i,0,n-1)to[i]=a[i];
    f(i,0,n-1)if(i<but[i])swap(to[i],to[but[i]]);

    for(int l=1;l<n;l<<=1){qwq++;
      ll Wn=qp((o==1?G:Gi),((mod-1)/(l<<1)));
      for(int i=1;i<l;i++)w[i]=w[i-1]*Wn%mod;
      for(int j=0;j<n;j+=l+l){
        for(int i=0;i<l;i++){
          ll tt=(__int128)w[i]*to[i|j|l]%mod;
          to[i|j|l]=mod+to[i|j]-tt;
          to[i|j]=to[i|j]+tt;
        }
      }if(qwq%9==0)f(i,0,n-1)to[i]%=mod;
    }if(o==1)inv=1;
    f(i,0,n-1)to[i]=(__int128)to[i]*inv%mod;
    return n-1;
  }
  void Mul(ll *f,ll *g,ll *to,ll n,ll m){
    static ll _f[PoL],_g[PoL],_n,_m;_n=n,_m=m;
    f(i,0,_n)_f[i]=f[i];
    f(i,0,_m)_g[i]=g[i];
```

```
  _m+=_n,_n=1;
  _n=NTT(_f,_f,_m,1);NTT(_g,_g,_m,1);
  f(i,0,_n)to[i]=_f[i]*_g[i]%mod;
  NTT(to,to,_m,-1);
  f(i,_m+1,_n)to[i]=0;
  f(i,0,_n)_f[i]=_g[i]=0;
}
void Inv(ll *a,ll *to,ll *usl,ll n){//a!=to
  ll k=1;while(k<=n)k<<=1;
      f(i,0,2*k-1)to[i]=usl[i]=0;
      k=1;to[0]=qp(a[0]);
  while(k<=n){k<<=1;
    f(i,0,k-1)usl[i]=(i<=n?a[i]:0);
          f(i,k,2*k-1)usl[i]=0;
    NTT(to,to,k*2-1,1),NTT(usl,usl,k*2-1,1);
    f(i,0,k*2-1)to[i]=(2-usl[i]*to[i]%mod+mod)%mod*to[i]%mod;
    NTT(to,to,k*2-1,-1);f(i,k,k*2-1)to[i]=0;
  }
}
void Dev(ll *a,ll *to,ll n){
  f(i,0,n-1)to[i]=a[i+1]*(i+1)%mod;to[n]=0;
}
void Int(ll *a,ll *to,ll n){pre();
  for(int i=n+1;i>=1;i--)to[i]=a[i-1]*inv[i]%mod;
  to[0]=0;//to[0]=原函数的常数项
}
void Ln(ll *a,ll *to,ll *usl,ll n){//a!=to
  f(i,1,4*n)usl[i]=0;
  Inv(a,to,usl,n);
  f(i,0,n)usl[i]=a[i];Dev(a,a,n);
  ll len=NTT(to,to,2*n,1);NTT(a,a,2*n,1);
  f(i,0,len)to[i]=to[i]*a[i]%mod;
  NTT(to,to,2*n,-1);f(i,0,n)a[i]=usl[i];
  Int(to,to,len);
  f(i,1,4*n)usl[i]=0;
}
void work(ll *a,ll *to,ll *f,ll *g,ll l,ll r){
  if(l==r){
    to[l]=to[l]*inv[l]%mod;
    return;
  }ll mid=l+r>>1;
  work(a,to,f,g,l,mid);ll lf=-1,lg=-1;
  f(i,l,mid)f[++lf]=to[i];f(i,0,r-1-l)g[++lg]=a[i];
  ll len=NTT(f,f,lf+lg,1);NTT(g,g,lf+lg,1);
  f(i,0,len)f[i]=f[i]*g[i]%mod;
  NTT(f,f,lf+lg,-1);
  f(i,mid+1,r)to[i]=(to[i]+f[i-1-l])%mod;
  f(i,0,len)f[i]=g[i]=0;
  work(a,to,f,g,mid+1,r);
}void exp(ll *a,ll *to,ll *f,ll *g,ll n){//a!=to
  f(i,0,4*n)f[i]=g[i]=to[i]=0;
  pre();
  Dev(a,a,n);to[0]=1;
  work(a,to,f,g,0,n);
}
ll lshift(ll *a,ll n,ll t){//a[t]->a[0]
  f(i,0,n-t)a[i]=a[i+t];
  return n-t;
}ll rshift(ll *a,ll n,ll t){//a[0]->a[t]
```

```
    for(int i=n+t;i>=t;i--)a[i]=a[i-t];
    f(i,0,t-1)a[i]=0;
    return n+t;
  }void mul(ll *a,ll n,ll k){
    f(i,0,n)a[i]=a[i]*k%mod;
  }
  void pwr(ll *a,ll *to,ll *f,ll *g,ll *h,ll n,ll m){//a!=to
    f(i,0,4*n)to[i]=f[i]=g[i]=h[i]=0;
        if(m==0){to[0]=1;return;}
    ll t=0;
    if(a[0]==0){
      f(i,0,n)if(a[i]!=0){t=i;break;}
      if(t==0||(__int128)m*t>n)return;
    }n=lshift(a,n,t);
    ll C=a[0],iv=qp(C);
    mul(a,n,iv);Ln(a,h,f,n);
    mul(h,n,m);exp(h,to,f,g,n);
    iv=qp(C,m);mul(to,n,iv);
    n=rshift(to,n,t*m);n=n-t*m+t;
  }
  void div(ll *f,ll *g,ll *gr,ll *gg,ll *ff,ll *Q,ll *R,ll n,ll m){
    f(i,0,4*n)gr[i]=ff[i]=0;
    reverse(f,f+n+1);reverse(g,g+m+1);
    Inv(g,gr,ff,n-m+1);
    ll len=NTT(f,ff,2*n,1);NTT(gr,gg,len,1);
    f(i,0,len)ff[i]=ff[i]*gg[i]%mod;
    NTT(ff,Q,len,-1);reverse(Q,Q+n-m+1);
    f(i,n-m+1,len)Q[i]=0;
    reverse(g,g+m+1);
    len=NTT(Q,ff,n*2,1);NTT(g,gr,n*2,1);
    f(i,0,len)ff[i]=ff[i]*gr[i]%mod;
    NTT(ff,ff,len,-1);
    reverse(f,f+n+1);
    f(i,0,n)R[i]=(f[i]-ff[i]+mod)%mod;
  }
}
using namespace poly;
ll f[PoL],g[PoL],ff[PoL],gg[PoL],hh[PoL];
ll n,m,mm,lim;
int main(){
  return 0;
}
```

vector 精简版（更新至 Inv）

```
ll but[(1<<21)+5],w[(1<<21)+5];
const ll mod=998244353,G=3,Gi=332748118;
ll qp(ll a,ll b=mod-2){ll ans=1;
  while(b){
    if(b&1)ans=ans*a%mod;
    a=a*a%mod;b>>=1;
  }return ans%mod;
}
void revbit(ll k){
    f(i,0,k-1)but[i]=but[i>>1]>>1|((i&1)?(k>>1):0);
}
#define ck(x) ((x)>=mod?(x)-mod:(x))
void NTT(vector<ll> &a,ll o){
    ll n=a.size()-1;
    ll _n=1;while(_n<=n)_n<<=1;
```

```
    n=_n;a.resize(n,0);

    revbit(n);ll qwq=0;
  ll inv=qp(n);
    f(i,0,n-1)if(i<but[i])swap(a[i],a[but[i]]);
    // for(auto _:a)cout<<_<<" ";cout<<endl;
    for(int l=1;l<n;l<<=1){qwq++;
        ll Wn=qp((o==1?G:Gi),((mod-1)/(l<<1)));
        w[0]=1;for(int i=1;i<l;i++)w[i]=w[i-1]*Wn%mod;
        for(int j=0;j<n;j+=l+l){
            for(int i=0;i<l;i++){
                ll tt=(__int128)w[i]*a[i|j|l]%mod;
                a[i|j|l]=mod+a[i|j]-tt;
                a[i|j]=a[i|j]+tt;
            }
        }if(qwq==9){
            qwq=0;
            f(i,0,n-1)a[i]%=mod;
        }
    }if(o==1)inv=1;
    f(i,0,n-1)a[i]=a[i]%mod*inv%mod;
}
vector<ll> Mul(vector<ll> a,vector<ll>b){
    if(a.empty()||b.empty())return {};
    ll n=a.size()-1,m=b.size()-1;
    a.resize(n+m+1,0),b.resize(n+m+1,0);
    NTT(a,1);NTT(b,1);
    f(i,0,a.size()-1)a[i]=a[i]*b[i]%mod;
    NTT(a,-1);a.resize(n+m+1);
    return a;
}

vector<ll> Inv(vector<ll> &a){
    if(a.empty()||a[0]==0){
        cerr<<"Err inverse don't exist.\n";
        assert(0);
    }
    int n=a.size();
    vector<ll>R(1,qp(a[0]));
    for (int k=1;k<n;k<<=1) {
        vector<ll>Q(a.begin(),a.begin()+min(n,k*2));
        vector<ll>S=R;
        S.resize(k*4,0);
        Q.resize(k*4,0);
        NTT(S,1);NTT(Q,1);
        for (int i=0;i<k*4;i++)
            S[i]=S[i]*(2+mod-S[i]*Q[i]%mod)%mod;
        NTT(S,-1);
        R.assign(S.begin(),S.begin()+k*2);
    }
    R.resize(n);
    return R;
}
```

多测与注意　完整恢复原指针版与 vector 精简版，n 仍按原来表示最高次数。保留原函数名与实参。指针版多测 / 多次运算前，输入的高次填充及输出、工作数组需清零到本次 NTT 所用长度；Ln、exp、pwr、div 会改输入，需复用则先复制。缓冲区须留够中间变换及积分空间；原大长度工作区管

理仍建议按题目检查，未重写。NTT 模数 998244353，长度不超过 2^{23}。

5.4.1 Bostan-Mori

求 $[z^m]\frac{P(z)}{Q(z)}$，原理是 $\frac{P(z)}{Q(z)}=\frac{P(z)Q(-z)}{Q(z)Q(-z)}$，随后分母只有偶次项，分子可以奇偶分开，根据 m 的奇偶性只需要取一边即可变成 $[z^{m/2}]\frac{P_1(z)}{Q_1(z)}$ 的一个子问题。

```
while(m){
  vector<ll>nQ;
  nQ.clear();
  int q=Q.size();
  f(i,0,q-1){
    if(i&1)nQ.pb((mod-Q[i])%mod);
    else nQ.pb(Q[i]);
  }
  // for(auto w:Q)cout<<w<<" ";cout<<endl;
  // for(auto w:nQ)cout<<w<<" ";cout<<endl;
  vector<ll> dP=Mul(P,nQ);
  vector<ll> dQ=Mul(Q,nQ);
  P.clear(),Q.clear();
  int dq=dQ.size();
  f(i,0,dq-1)
    if(i%2==0)Q.pb(dQ[i]);
  int dp=dP.size();
  f(i,0,dp-1)
    if(i%2==m%2)P.pb(dP[i]);
  m>>=1;
}
printf("%lld\n",P.empty()?0:P[0]*qp(Q[0])%mod);
```

多测与注意 保留原循环，P、Q、m 每组重读，Q[0] 非零。代码会破坏 P、Q；零分子输出已补。

5.5 数值方法 (Numerical Methods)

5.5.1 自适应辛普森 (Adaptive Simpson's Rule)

自适应辛普森积分，用于数值计算函数在区间上的定积分。

```
double f(double x){return (m*x+n)/(a*x+b);}//在x处的函数值
double simp(double L,double R){return ((R-L)*(f(L)+f(R)+4*f((L+R)/2)))/6;}
double asr(double L,double R,double eps,double ans){
  double mid=(L+R)/2;
  double x=simp(L,mid),y=simp(mid,R);
  if(mid==L||mid==R)return ans;
  if(fabs(x+y-ans)<=15*eps)return x+y+(x+y-ans)/15;
  return asr(L,mid,eps/2,x)+asr(mid,R,eps/2,y);
}

printf("%.6lf",asr(l,r,1e-6,simp(l,r)));
```

多测与注意 保留原 f、simp、asr。每组更新函数参数，无缓存要清；区间不能有极点，精度与可能的递归过深按题目处理。

6 Misc

6.1 C++ 手动上下取整

众所周知 c++ 的负数取整逻辑是向 0 取整，强需要向下/上取整的时候很麻烦。

```
ll Ceil(ll x,ll y);
ll Floor(ll x,ll y){
    ll q=x/y,r=x%y;return q-(r!=0&&((r<0)!=(y<0)));
}ll Ceil(ll x,ll y){
    ll q=x/y,r=x%y;return q+(r!=0&&((r<0)==(y<0)));
}
```

多测与注意 函数名不变，只改商余数修正，避免 x+y-1 和取负溢出。除数非零，商可由 ll 表示。

6.2 单点修改区间摩尔投票

线段树维护摩尔投票过程，节点 (c, num) 表示投票完以后剩下的数和个数（没有的话记为 -1）。合并时若候选不同，保留计数较大的候选，计数取两者之差（相等则为 0）；候选相同才相加。

然后还有一步要检验出来的数是不是合法，需要维护一个区间出现某个数的次数。可以平衡树。

多测与注意 候选需再验证真实出现次数。每组重建线段树及位置集合，修改同步更新旧值、新值的位置。

6.3 双栈模拟 dequeue

两个栈，左边的一个维护当前位置到栈底的前缀操作和，右边一个维护栈底到当前位置的前缀操作和。查询就是两个栈顶的维护值操作一下，首尾 add 都是简单的。

困难的在于某个栈空了的时候的删除操作。把另一个栈的元素切成等大小的左右两段，暴力重构即可均摊 $O(1)$。

多测与注意 每组清双栈和聚合缓存；非交换操作注意左右累积方向。

6.4 测速

```
auto start = std::chrono::steady_clock::now();

// main code here

auto end = std::chrono::steady_clock::now();
std::chrono::duration<double> elapsed = end - start;
std::cerr << "\nRunning: " << std::chrono::duration_cast<std::chrono::milliseconds>(end - start).
    count() << " ms" << std::endl;
```

多测与注意 测单组就把计时放 solve 内，测整体则放多测循环外。

6.5 KKT

$$\begin{aligned} \min_{x\in\mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \le 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

KKT:

1. Stationarity: $\nabla f(x^*) + \sum_{i=1}^{m} \lambda_i^* \nabla g_i(x^*) + \sum_{j=1}^{p} \nu_j^* \nabla h_j(x^*) = 0$
2. Primal Feasibility: $g_i(x^*) \le 0, \quad h_j(x^*) = 0$
3. Dual Feasibility: $\lambda_i^* \ge 0$
4. Complementary Slackness: $\lambda_i^* g_i(x^*) = 0$

多测与注意 非凸问题满足 KKT 不保证全局最优；必要性需约束规范性。凸问题满足 KKT 时充分。